

# 第四讲 感知与通信协议

责任助教：姚遥 刘若桐



# 01 串口通信



# 目录

## 1.基本概念

- 串口，串口通信，特性

## 2.ESP32-S3中的UART资源及其运用

- 硬件连接、代码实现
- 演示实验一：通过板载USB接口实现与PC的简单通信
- 演示实验二：通过USB-TTL实现单片机与PC的通信
- 演示实验三：UART实现两单片机间通信

## 3.OLED的使用（I2C协议的应用实例，I2C必修不要求）

- 硬件连接、调库

# 1.基本概念

- **串口**：**串行**接口（硬件接口）——今天要讲的UART、选修中要讲的I2C都是串行口
- **串口通信**：串口按位（bit）发送和接收字节的**通信方式**
- **分类**——同步与异步、点对点与主从等

## 常见的通信方式—— UART和I2C

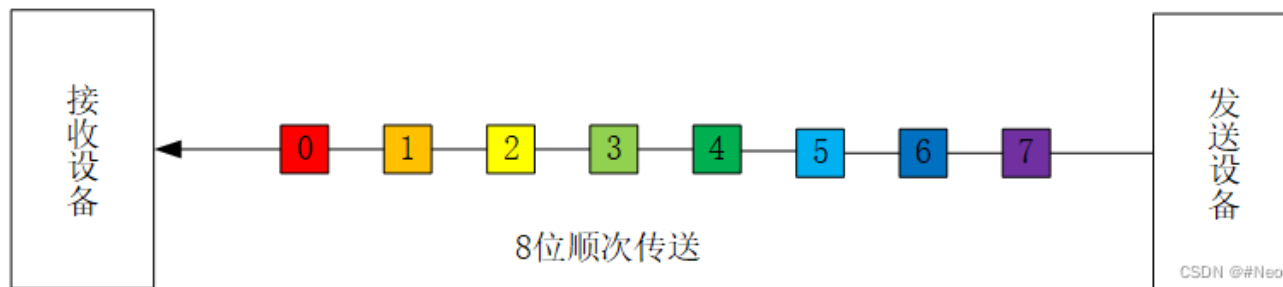
- |                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>■ <b>UART</b> (Universal Asynchronous Receiver and Transmitter)</li><li>■ <b>全称</b>：通用异步接收器/发射器</li><li>■ <b>特点</b>：<ul style="list-style-type: none"><li>串行</li><li>异步</li><li>全双工</li></ul></li><li>■ <b>应用</b>：串口调试（开发板与PC之间的数据收发）、与传感器等模块通信，等等</li></ul> | <ul style="list-style-type: none"><li>■ I2C 或IIC (Inter-Integrated Circuit) // 选修“通信协议进阶”会详细讲解</li><li>■ <b>特点</b>：<ul style="list-style-type: none"><li>串行</li><li>同步</li><li>半双工</li><li>总线</li></ul></li><li>■ <b>应用</b>：与模块通信，如<b>OLED</b>、MPU等</li></ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# 1.基本概念

- 串口：串行接口（硬件接口）——今天要讲的UART、选修中要讲的I2C都是串行口
- 串口通信：串口按位（bit）发送和接收字节的通信方式
- 分类——同步与异步、点对点与主从等

## ①串行与并行

串行通信



并行通信



# 1.基本概念

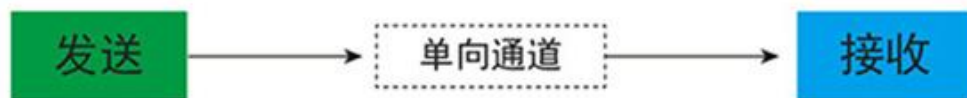
- 串口：串行接口（硬件接口）
- 串口通信：串口按位（bit）发送和接收字节的通信方式
- 分类——同步与异步、点对点与主从等

## ②单工、全双工、半双工

### 单工（Simplex）

- 只支持信号在一个方向上传输(正向或反向), 任何时候不能改变信号的传输方向。

• 单工通信方式



### 半双工（Half-Duplex）

- 允许信号在两个方向上传输，但某一时刻只允许信号在一个信道上单向传输。因此，半双工通信实际上是一种可切换方向的单工通信。

• 半双工通信方式



### 全双工（Full-Duplex） UART支持全双工

- 允许数据同时在两个方向上传输，即有两个信道，因此允许同时进行双向传输。是两个单工通信方式的结合，要求收发双方都有独立的接收和发送能力。

• 全双工通信方式



# 1.基本概念

- 串口：串行接口（硬件接口）
- 串口通信：串口按位（bit）发送和接收字节的通信方式
- 分类——同步与异步、点对点与主从等

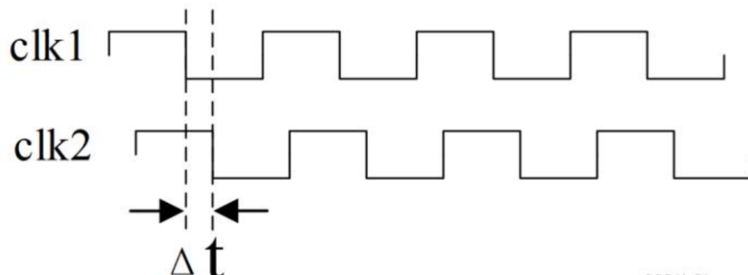
## ③同步与异步 UART为异步通信

同步 Synchronous

- 通信双方共享一个时钟信号来协调数据传输节奏。

异步 Asynchronous

- 通信双方没有共享时钟，依靠预定义的规则（波特率、起止位）来协调每个数据块的开始和结束。



**Baud Rate**，表示单位时间内传输的符号数，在串口通信中，波特率指的是每秒钟传输的二进制位数，常见波特率有9600、115200等。通信双方必须设定相同的波特率，否则无法正确接收数据。

# 1.基本概念

- 串口：串行接口（硬件接口）
- 串口通信：串口按位（bit）发送和接收字节的通信方式
- 分类——同步与异步、点对点与主从等

## UART的异步通信——帧结构

**数据帧（Data Frame）**：数据以帧为单位进行传

输。每个UART数据帧包含：

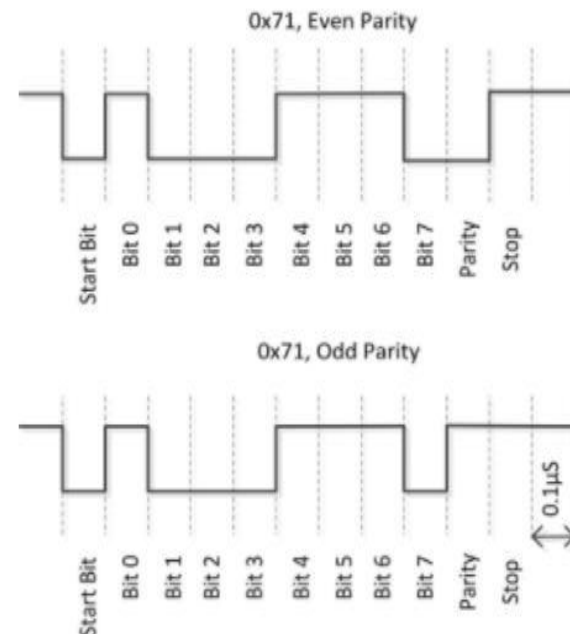
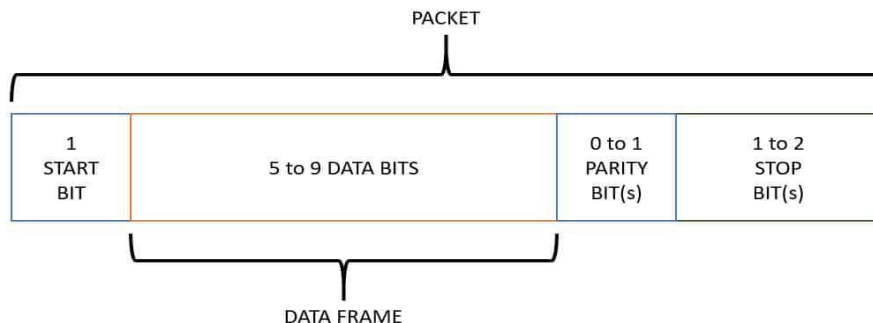
**起始位**：1bit，低电平，表示数据传输开始。

**数据位**：通常是8 bits（5~9），实际传输的数据。

**校验位（可无）**：1bit，用于简单的错误检测。

**停止位**：1~2bit，高电平，表示数据传输结束。

每位持续的时间是固定的，由发送器本地时钟控制，每秒发送的数据位个数，就是“波特率”。





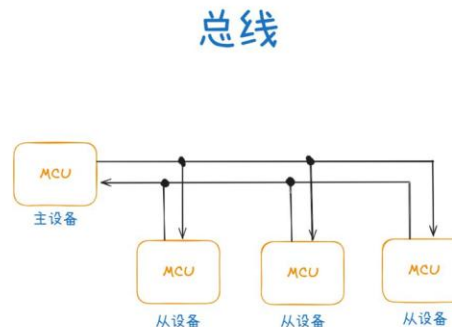
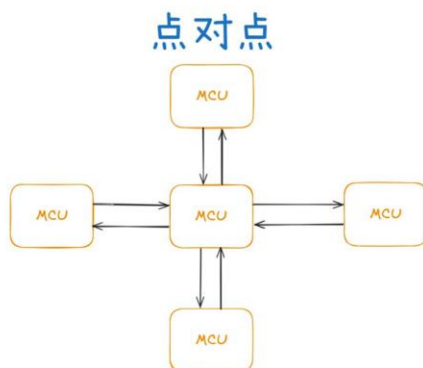
# 1.基本概念

- 串口：串行接口（硬件接口）
- 串口通信：串口按位（bit）发送和接收字节的通信方式
- 分类——同步与异步、点对点与主从等

## ④点对点与总线

UART默认为点对点

- 点对点
  - 两个设备独占一条专用通信线路。（畅通）
- 总线
  - 多个设备共享一组公共通信线路。
  - 一般有一个主设备(Master)+多个从设备(Slave)



## 2.ESP32-S3中的UART资源及其运用

表 2-9. 外设管脚分配

| 管脚序号 | 管脚名称       | USB 串1/JTAG <sup>1</sup> | USB OTG <sup>1</sup> | JTAG      | ADC1          | ADC2 | 触摸传感器        | UART0 <sup>2</sup> | UART1 <sup>2</sup> | SPI0/1 (推荐) <sup>2</sup> | SPI0/1 (备选) <sup>2</sup> | SPI2 (推荐) <sup>2</sup> | SPI2 (备选) <sup>2</sup> | UART2       | I2S         | LED PWM     | I2S         | LCD & Camera | SPI3        | SD/MMC      | MCU PWM     | RMT         | PCNT        |
|------|------------|--------------------------|----------------------|-----------|---------------|------|--------------|--------------------|--------------------|--------------------------|--------------------------|------------------------|------------------------|-------------|-------------|-------------|-------------|--------------|-------------|-------------|-------------|-------------|-------------|
| 1    | LNA_IN     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 2    | VDD3P3     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 3    | VDD3P3     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 4    | CHIP_PU    |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 5    | GPIO0      |                          |                      |           |               |      |              | GPIO0 (P3)         | GPIO0 (P3)         | GPIO0 (P3)               | GPIO0 (P3)               | GPIO0 (P3)             | GPIO0 (P3)             | GPIO0 (P3)  | GPIO0 (P3)  | GPIO0 (P3)  | GPIO0 (P3)  | GPIO0 (P3)   | GPIO0 (P3)  | GPIO0 (P3)  | GPIO0 (P3)  | GPIO0 (P3)  | GPIO0 (P3)  |
| 6    | GPIO1      |                          |                      |           | ADC1_CH0 (P1) |      | TOUCH1 (P1)  | GPIO1 (P2)         | GPIO1 (P2)         | GPIO1 (P2)               | GPIO1 (P2)               | GPIO1 (P2)             | GPIO1 (P2)             | GPIO1 (P2)  | GPIO1 (P2)  | GPIO1 (P2)  | GPIO1 (P2)  | GPIO1 (P2)   | GPIO1 (P2)  | GPIO1 (P2)  | GPIO1 (P2)  | GPIO1 (P2)  | GPIO1 (P2)  |
| 7    | GPIO2      |                          |                      |           | ADC1_CH1 (P1) |      | TOUCH2 (P1)  | GPIO2 (P2)         | GPIO2 (P2)         | GPIO2 (P2)               | GPIO2 (P2)               | GPIO2 (P2)             | GPIO2 (P2)             | GPIO2 (P2)  | GPIO2 (P2)  | GPIO2 (P2)  | GPIO2 (P2)  | GPIO2 (P2)   | GPIO2 (P2)  | GPIO2 (P2)  | GPIO2 (P2)  | GPIO2 (P2)  | GPIO2 (P2)  |
| 8    | GPIO3      |                          |                      |           | ADC1_CH2 (P1) |      | TOUCH3 (P1)  | GPIO3 (P3)         | GPIO3 (P3)         | GPIO3 (P3)               | GPIO3 (P3)               | GPIO3 (P3)             | GPIO3 (P3)             | GPIO3 (P3)  | GPIO3 (P3)  | GPIO3 (P3)  | GPIO3 (P3)  | GPIO3 (P3)   | GPIO3 (P3)  | GPIO3 (P3)  | GPIO3 (P3)  | GPIO3 (P3)  | GPIO3 (P3)  |
| 9    | GPIO4      |                          |                      |           | ADC1_CH3 (P1) |      | TOUCH4 (P1)  | GPIO4 (P2)         | GPIO4 (P2)         | GPIO4 (P2)               | GPIO4 (P2)               | GPIO4 (P2)             | GPIO4 (P2)             | GPIO4 (P2)  | GPIO4 (P2)  | GPIO4 (P2)  | GPIO4 (P2)  | GPIO4 (P2)   | GPIO4 (P2)  | GPIO4 (P2)  | GPIO4 (P2)  | GPIO4 (P2)  | GPIO4 (P2)  |
| 10   | GPIO5      |                          |                      |           | ADC1_CH4 (P1) |      | TOUCH5 (P1)  | GPIO5 (P2)         | GPIO5 (P2)         | GPIO5 (P2)               | GPIO5 (P2)               | GPIO5 (P2)             | GPIO5 (P2)             | GPIO5 (P2)  | GPIO5 (P2)  | GPIO5 (P2)  | GPIO5 (P2)  | GPIO5 (P2)   | GPIO5 (P2)  | GPIO5 (P2)  | GPIO5 (P2)  | GPIO5 (P2)  | GPIO5 (P2)  |
| 11   | GPIO6      |                          |                      |           | ADC1_CH5 (P1) |      | TOUCH6 (P1)  | GPIO6 (P2)         | GPIO6 (P2)         | GPIO6 (P2)               | GPIO6 (P2)               | GPIO6 (P2)             | GPIO6 (P2)             | GPIO6 (P2)  | GPIO6 (P2)  | GPIO6 (P2)  | GPIO6 (P2)  | GPIO6 (P2)   | GPIO6 (P2)  | GPIO6 (P2)  | GPIO6 (P2)  | GPIO6 (P2)  | GPIO6 (P2)  |
| 12   | GPIO7      |                          |                      |           | ADC1_CH6 (P1) |      | TOUCH7 (P1)  | GPIO7 (P2)         | GPIO7 (P2)         | GPIO7 (P2)               | GPIO7 (P2)               | GPIO7 (P2)             | GPIO7 (P2)             | GPIO7 (P2)  | GPIO7 (P2)  | GPIO7 (P2)  | GPIO7 (P2)  | GPIO7 (P2)   | GPIO7 (P2)  | GPIO7 (P2)  | GPIO7 (P2)  | GPIO7 (P2)  | GPIO7 (P2)  |
| 13   | GPIO8      |                          |                      |           | ADC1_CH7 (P1) |      | TOUCH8 (P1)  | GPIO8 (P2)         | GPIO8 (P2)         | GPIO8 (P2)               | SUBSPIC1 (P1)            | GPIO8 (P2)             | GPIO8 (P2)             | GPIO8 (P2)  | GPIO8 (P2)  | GPIO8 (P2)  | GPIO8 (P2)  | GPIO8 (P2)   | GPIO8 (P2)  | GPIO8 (P2)  | GPIO8 (P2)  | GPIO8 (P2)  | GPIO8 (P2)  |
| 14   | GPIO9      |                          |                      |           | ADC1_CH8 (P1) |      | TOUCH9 (P1)  | GPIO9 (P2)         | GPIO9 (P2)         | GPIO9 (P2)               | SUBSPIC2 (P1)            | GPIO9 (P2)             | GPIO9 (P2)             | GPIO9 (P2)  | GPIO9 (P2)  | GPIO9 (P2)  | GPIO9 (P2)  | GPIO9 (P2)   | GPIO9 (P2)  | GPIO9 (P2)  | GPIO9 (P2)  | GPIO9 (P2)  | GPIO9 (P2)  |
| 15   | GPIO10     |                          |                      |           | ADC1_CH9 (P1) |      | TOUCH10 (P1) | GPIO10 (P2)        | GPIO10 (P2)        | GPIO10 (P2)              | SUBSPIC3 (P1)            | GPIO10 (P2)            | GPIO10 (P2)            | GPIO10 (P2) | GPIO10 (P2) | GPIO10 (P2) | GPIO10 (P2) | GPIO10 (P2)  | GPIO10 (P2) | GPIO10 (P2) | GPIO10 (P2) | GPIO10 (P2) | GPIO10 (P2) |
| 16   | GPIO11     |                          |                      |           | ADC2_CH0 (P1) |      | TOUCH11 (P1) | GPIO11 (P2)        | GPIO11 (P2)        | GPIO11 (P2)              | SUBSPIC4 (P1)            | GPIO11 (P2)            | GPIO11 (P2)            | GPIO11 (P2) | GPIO11 (P2) | GPIO11 (P2) | GPIO11 (P2) | GPIO11 (P2)  | GPIO11 (P2) | GPIO11 (P2) | GPIO11 (P2) | GPIO11 (P2) | GPIO11 (P2) |
| 17   | GPIO12     |                          |                      |           | ADC2_CH1 (P1) |      | TOUCH12 (P1) | GPIO12 (P2)        | GPIO12 (P2)        | GPIO12 (P2)              | SUBSPIC5 (P1)            | GPIO12 (P2)            | GPIO12 (P2)            | GPIO12 (P2) | GPIO12 (P2) | GPIO12 (P2) | GPIO12 (P2) | GPIO12 (P2)  | GPIO12 (P2) | GPIO12 (P2) | GPIO12 (P2) | GPIO12 (P2) | GPIO12 (P2) |
| 18   | GPIO13     |                          |                      |           | ADC2_CH2 (P1) |      | TOUCH13 (P1) | GPIO13 (P2)        | GPIO13 (P2)        | GPIO13 (P2)              | SUBSPIC6 (P1)            | GPIO13 (P2)            | GPIO13 (P2)            | GPIO13 (P2) | GPIO13 (P2) | GPIO13 (P2) | GPIO13 (P2) | GPIO13 (P2)  | GPIO13 (P2) | GPIO13 (P2) | GPIO13 (P2) | GPIO13 (P2) | GPIO13 (P2) |
| 19   | GPIO14     |                          |                      |           | ADC2_CH3 (P1) |      | TOUCH14 (P1) | GPIO14 (P2)        | GPIO14 (P2)        | GPIO14 (P2)              | SUBSPIC7 (P1)            | GPIO14 (P2)            | GPIO14 (P2)            | GPIO14 (P2) | GPIO14 (P2) | GPIO14 (P2) | GPIO14 (P2) | GPIO14 (P2)  | GPIO14 (P2) | GPIO14 (P2) | GPIO14 (P2) | GPIO14 (P2) | GPIO14 (P2) |
| 20   | VDD3P3_RTC |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 21   | XTAL_32K_P |                          |                      |           | ADC2_CH4 (P1) |      |              | GPIO15 (P2)        | GPIO15 (P2)        | GPIO15 (P2)              | GPIO15 (P2)              | GPIO15 (P2)            | GPIO15 (P2)            | GPIO15 (P2) | GPIO15 (P2) | GPIO15 (P2) | GPIO15 (P2) | GPIO15 (P2)  | GPIO15 (P2) | GPIO15 (P2) | GPIO15 (P2) | GPIO15 (P2) | GPIO15 (P2) |
| 22   | XTAL_32K_N |                          |                      |           |               |      |              | GPIO16 (P2)        | GPIO16 (P2)        | GPIO16 (P2)              | GPIO16 (P2)              | GPIO16 (P2)            | GPIO16 (P2)            | GPIO16 (P2) | GPIO16 (P2) | GPIO16 (P2) | GPIO16 (P2) | GPIO16 (P2)  | GPIO16 (P2) | GPIO16 (P2) | GPIO16 (P2) | GPIO16 (P2) | GPIO16 (P2) |
| 23   | GPIO17     |                          |                      |           | ADC2_CH5 (P1) |      |              | GPIO17 (P2)        | GPIO17 (P2)        | GPIO17 (P2)              | GPIO17 (P2)              | GPIO17 (P2)            | GPIO17 (P2)            | GPIO17 (P2) | GPIO17 (P2) | GPIO17 (P2) | GPIO17 (P2) | GPIO17 (P2)  | GPIO17 (P2) | GPIO17 (P2) | GPIO17 (P2) | GPIO17 (P2) | GPIO17 (P2) |
| 24   | GPIO18     |                          |                      |           | ADC2_CH6 (P1) |      |              | GPIO18 (P2)        | GPIO18 (P2)        | GPIO18 (P2)              | GPIO18 (P2)              | GPIO18 (P2)            | GPIO18 (P2)            | GPIO18 (P2) | GPIO18 (P2) | GPIO18 (P2) | GPIO18 (P2) | GPIO18 (P2)  | GPIO18 (P2) | GPIO18 (P2) | GPIO18 (P2) | GPIO18 (P2) | GPIO18 (P2) |
| 25   | GPIO19     | USB_D- (P1)              | USB_D+ (P1)          |           | ADC2_CH7 (P1) |      |              | GPIO19 (P3)        | GPIO19 (P3)        | GPIO19 (P3)              | GPIO19 (P3)              | GPIO19 (P3)            | GPIO19 (P3)            | GPIO19 (P3) | GPIO19 (P3) | GPIO19 (P3) | GPIO19 (P3) | GPIO19 (P3)  | GPIO19 (P3) | GPIO19 (P3) | GPIO19 (P3) | GPIO19 (P3) | GPIO19 (P3) |
| 26   | GPIO20     | USB_D+ (P1)              | USB_D- (P1)          |           | ADC2_CH8 (P1) |      |              | GPIO20 (P3)        | GPIO20 (P3)        | GPIO20 (P3)              | GPIO20 (P3)              | GPIO20 (P3)            | GPIO20 (P3)            | GPIO20 (P3) | GPIO20 (P3) | GPIO20 (P3) | GPIO20 (P3) | GPIO20 (P3)  | GPIO20 (P3) | GPIO20 (P3) | GPIO20 (P3) | GPIO20 (P3) | GPIO20 (P3) |
| 27   | GPIO21     | GPIO21 (P2)              | GPIO21 (P2)          |           |               |      |              | GPIO21 (P2)        | GPIO21 (P2)        | GPIO21 (P2)              | GPIO21 (P2)              | GPIO21 (P2)            | GPIO21 (P2)            | GPIO21 (P2) | GPIO21 (P2) | GPIO21 (P2) | GPIO21 (P2) | GPIO21 (P2)  | GPIO21 (P2) | GPIO21 (P2) | GPIO21 (P2) | GPIO21 (P2) | GPIO21 (P2) |
| 28   | SPICS1     |                          |                      |           |               |      |              | GPIO26 (P4)        | GPIO26 (P4)        | GPIO26 (P4)              | GPIO26 (P4)              | GPIO26 (P4)            | GPIO26 (P4)            | GPIO26 (P4) | GPIO26 (P4) | GPIO26 (P4) | GPIO26 (P4) | GPIO26 (P4)  | GPIO26 (P4) | GPIO26 (P4) | GPIO26 (P4) | GPIO26 (P4) | GPIO26 (P4) |
| 29   | VDD_SPI    |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 30   | SPH0       |                          |                      |           |               |      |              | GPIO27 (P4)        | GPIO27 (P4)        | GPIO27 (P4)              | GPIO27 (P4)              | GPIO27 (P4)            | GPIO27 (P4)            | GPIO27 (P4) | GPIO27 (P4) | GPIO27 (P4) | GPIO27 (P4) | GPIO27 (P4)  | GPIO27 (P4) | GPIO27 (P4) | GPIO27 (P4) | GPIO27 (P4) | GPIO27 (P4) |
| 31   | SPH1       |                          |                      |           |               |      |              | GPIO28 (P4)        | GPIO28 (P4)        | GPIO28 (P4)              | GPIO28 (P4)              | GPIO28 (P4)            | GPIO28 (P4)            | GPIO28 (P4) | GPIO28 (P4) | GPIO28 (P4) | GPIO28 (P4) | GPIO28 (P4)  | GPIO28 (P4) | GPIO28 (P4) | GPIO28 (P4) | GPIO28 (P4) | GPIO28 (P4) |
| 32   | SPH2       |                          |                      |           |               |      |              | GPIO29 (P4)        | GPIO29 (P4)        | GPIO29 (P4)              | GPIO29 (P4)              | GPIO29 (P4)            | GPIO29 (P4)            | GPIO29 (P4) | GPIO29 (P4) | GPIO29 (P4) | GPIO29 (P4) | GPIO29 (P4)  | GPIO29 (P4) | GPIO29 (P4) | GPIO29 (P4) | GPIO29 (P4) | GPIO29 (P4) |
| 33   | SPH3       |                          |                      |           |               |      |              | GPIO30 (P4)        | GPIO30 (P4)        | GPIO30 (P4)              | GPIO30 (P4)              | GPIO30 (P4)            | GPIO30 (P4)            | GPIO30 (P4) | GPIO30 (P4) | GPIO30 (P4) | GPIO30 (P4) | GPIO30 (P4)  | GPIO30 (P4) | GPIO30 (P4) | GPIO30 (P4) | GPIO30 (P4) | GPIO30 (P4) |
| 34   | SPH4       |                          |                      |           |               |      |              | GPIO31 (P4)        | GPIO31 (P4)        | GPIO31 (P4)              | GPIO31 (P4)              | GPIO31 (P4)            | GPIO31 (P4)            | GPIO31 (P4) | GPIO31 (P4) | GPIO31 (P4) | GPIO31 (P4) | GPIO31 (P4)  | GPIO31 (P4) | GPIO31 (P4) | GPIO31 (P4) | GPIO31 (P4) | GPIO31 (P4) |
| 35   | SPH5       |                          |                      |           |               |      |              | GPIO32 (P4)        | GPIO32 (P4)        | GPIO32 (P4)              | GPIO32 (P4)              | GPIO32 (P4)            | GPIO32 (P4)            | GPIO32 (P4) | GPIO32 (P4) | GPIO32 (P4) | GPIO32 (P4) | GPIO32 (P4)  | GPIO32 (P4) | GPIO32 (P4) | GPIO32 (P4) | GPIO32 (P4) | GPIO32 (P4) |
| 36   | SPH6       |                          |                      |           |               |      |              | GPIO33 (P3)        | GPIO33 (P3)        | GPIO33 (P3)              | GPIO33 (P3)              | GPIO33 (P3)            | GPIO33 (P3)            | GPIO33 (P3) | GPIO33 (P3) | GPIO33 (P3) | GPIO33 (P3) | GPIO33 (P3)  | GPIO33 (P3) | GPIO33 (P3) | GPIO33 (P3) | GPIO33 (P3) | GPIO33 (P3) |
| 37   | SPH7       |                          |                      |           |               |      |              | GPIO34 (P3)        | GPIO34 (P3)        | GPIO34 (P3)              | GPIO34 (P3)              | GPIO34 (P3)            | GPIO34 (P3)            | GPIO34 (P3) | GPIO34 (P3) | GPIO34 (P3) | GPIO34 (P3) | GPIO34 (P3)  | GPIO34 (P3) | GPIO34 (P3) | GPIO34 (P3) | GPIO34 (P3) | GPIO34 (P3) |
| 38   | GPIO33     |                          |                      |           |               |      |              | GPIO35 (P3)        | GPIO35 (P3)        | GPIO35 (P3)              | GPIO35 (P3)              | GPIO35 (P3)            | GPIO35 (P3)            | GPIO35 (P3) | GPIO35 (P3) | GPIO35 (P3) | GPIO35 (P3) | GPIO35 (P3)  | GPIO35 (P3) | GPIO35 (P3) | GPIO35 (P3) | GPIO35 (P3) | GPIO35 (P3) |
| 39   | GPIO34     |                          |                      |           |               |      |              | GPIO36 (P3)        | GPIO36 (P3)        | GPIO36 (P3)              | GPIO36 (P3)              | GPIO36 (P3)            | GPIO36 (P3)            | GPIO36 (P3) | GPIO36 (P3) | GPIO36 (P3) | GPIO36 (P3) | GPIO36 (P3)  | GPIO36 (P3) | GPIO36 (P3) | GPIO36 (P3) | GPIO36 (P3) | GPIO36 (P3) |
| 40   | GPIO35     |                          |                      |           |               |      |              | GPIO37 (P3)        | GPIO37 (P3)        | GPIO37 (P3)              | GPIO37 (P3)              | GPIO37 (P3)            | GPIO37 (P3)            | GPIO37 (P3) | GPIO37 (P3) | GPIO37 (P3) | GPIO37 (P3) | GPIO37 (P3)  | GPIO37 (P3) | GPIO37 (P3) | GPIO37 (P3) | GPIO37 (P3) | GPIO37 (P3) |
| 41   | GPIO36     |                          |                      |           |               |      |              | GPIO38 (P2)        | GPIO38 (P2)        | GPIO38 (P2)              | GPIO38 (P2)              | GPIO38 (P2)            | GPIO38 (P2)            | GPIO38 (P2) | GPIO38 (P2) | GPIO38 (P2) | GPIO38 (P2) | GPIO38 (P2)  | GPIO38 (P2) | GPIO38 (P2) | GPIO38 (P2) | GPIO38 (P2) | GPIO38 (P2) |
| 42   | GPIO37     |                          |                      |           |               |      |              | GPIO39 (P3)        | GPIO39 (P3)        | GPIO39 (P3)              | GPIO39 (P3)              | GPIO39 (P3)            | GPIO39 (P3)            | GPIO39 (P3) | GPIO39 (P3) | GPIO39 (P3) | GPIO39 (P3) | GPIO39 (P3)  | GPIO39 (P3) | GPIO39 (P3) | GPIO39 (P3) | GPIO39 (P3) | GPIO39 (P3) |
| 43   | GPIO38     | GPIO38 (P2)              | GPIO38 (P2)          |           |               |      |              | GPIO40 (P3)        | GPIO40 (P3)        | GPIO40 (P3)              | GPIO40 (P3)              | GPIO40 (P3)            | GPIO40 (P3)            | GPIO40 (P3) | GPIO40 (P3) | GPIO40 (P3) | GPIO40 (P3) | GPIO40 (P3)  | GPIO40 (P3) | GPIO40 (P3) | GPIO40 (P3) | GPIO40 (P3) | GPIO40 (P3) |
| 44   | MTCK       | MTCK (P1)                | MTCK (P1)            | MTCK (P1) |               |      |              | GPIO41 (P3)        | GPIO41 (P3)        | GPIO41 (P3)              | GPIO41 (P3)              | GPIO41 (P3)            | GPIO41 (P3)            | GPIO41 (P3) | GPIO41 (P3) | GPIO41 (P3) | GPIO41 (P3) | GPIO41 (P3)  | GPIO41 (P3) | GPIO41 (P3) | GPIO41 (P3) | GPIO41 (P3) | GPIO41 (P3) |
| 45   | MTDO       | MTDO (P1)                | MTDO (P1)            | MTDO (P1) |               |      |              | GPIO42 (P3)        | GPIO42 (P3)        | GPIO42 (P3)              | GPIO42 (P3)              | GPIO42 (P3)            | GPIO42 (P3)            | GPIO42 (P3) | GPIO42 (P3) | GPIO42 (P3) | GPIO42 (P3) | GPIO42 (P3)  | GPIO42 (P3) | GPIO42 (P3) | GPIO42 (P3) | GPIO42 (P3) | GPIO42 (P3) |
| 46   | VDD3P3_CPU |                          |                      |           |               |      |              | GPIO43 (P3)        | GPIO43 (P3)        | GPIO43 (P3)              | GPIO43 (P3)              | GPIO43 (P3)            | GPIO43 (P3)            | GPIO43 (P3) | GPIO43 (P3) | GPIO43 (P3) | GPIO43 (P3) | GPIO43 (P3)  | GPIO43 (P3) | GPIO43 (P3) | GPIO43 (P3) | GPIO43 (P3) | GPIO43 (P3) |
| 47   | MTDI       | MTDI (P1)                | MTDI (P1)            | MTDI (P1) |               |      |              | GPIO44 (P3)        | GPIO44 (P3)        | GPIO44 (P3)              | GPIO44 (P3)              | GPIO44 (P3)            | GPIO44 (P3)            | GPIO44 (P3) | GPIO44 (P3) | GPIO44 (P3) | GPIO44 (P3) | GPIO44 (P3)  | GPIO44 (P3) | GPIO44 (P3) | GPIO44 (P3) | GPIO44 (P3) | GPIO44 (P3) |
| 48   | MTMS       | MTMS (P1)                | MTMS (P1)            | MTMS (P1) |               |      |              | GPIO45 (P3)        | GPIO45 (P3)        | GPIO45 (P3)              | GPIO45 (P3)              | GPIO45 (P3)            | GPIO45 (P3)            | GPIO45 (P3) | GPIO45 (P3) | GPIO45 (P3) | GPIO45 (P3) | GPIO45 (P3)  | GPIO45 (P3) | GPIO45 (P3) | GPIO45 (P3) | GPIO45 (P3) | GPIO45 (P3) |
| 49   | U0TXD      |                          |                      |           |               |      |              | GPIO46 (P3)        | GPIO46 (P3)        | GPIO46 (P3)              | GPIO46 (P3)              | GPIO46 (P3)            | GPIO46 (P3)            | GPIO46 (P3) | GPIO46 (P3) | GPIO46 (P3) | GPIO46 (P3) | GPIO46 (P3)  | GPIO46 (P3) | GPIO46 (P3) | GPIO46 (P3) | GPIO46 (P3) | GPIO46 (P3) |
| 50   | U0RXD      |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 51   | GPIO45     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 52   | GPIO46     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 53   | XTAL_N     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 54   | XTAL_P     |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 55   | VDDA       |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 56   | VDDA       |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |
| 57   | GND        |                          |                      |           |               |      |              |                    |                    |                          |                          |                        |                        |             |             |             |             |              |             |             |             |             |             |

<sup>1</sup> USB 串1/JTAG 和 USB OTG 连接内部 PHY 时使用 USB\_D- 和 USB\_D+，且 USB\_D- 和 USB\_D+ 管脚可按照《ESP32-S3 技术参考手册》的 USB\_SERIAL\_JTAG\_EXCHG\_PINS 位配置后交换；连接外部 PHY 时使用 其余固定管脚。关于如何选择 PHY，详见《ESP32-S3 技术参考手册》> 章节 USB 串1/JTAG 控制器 > 章节内部/外部 PHY 选择。

<sup>2</sup> UART0、UART1、SPI0/1、SPI2 接口的外设信号，无论是否通过 IO MUX 直接连接 固定管脚，均可通过 GPIO 交换矩阵映射至任意 GPIO 管脚。

## 2.ESP32-S3中的UART资源及其运用

外部IO

RTS CTS引脚（我们的课程不用接）

硬件流控（不要求）

缓冲区  
FIFO

外部IO  
RX TX引脚

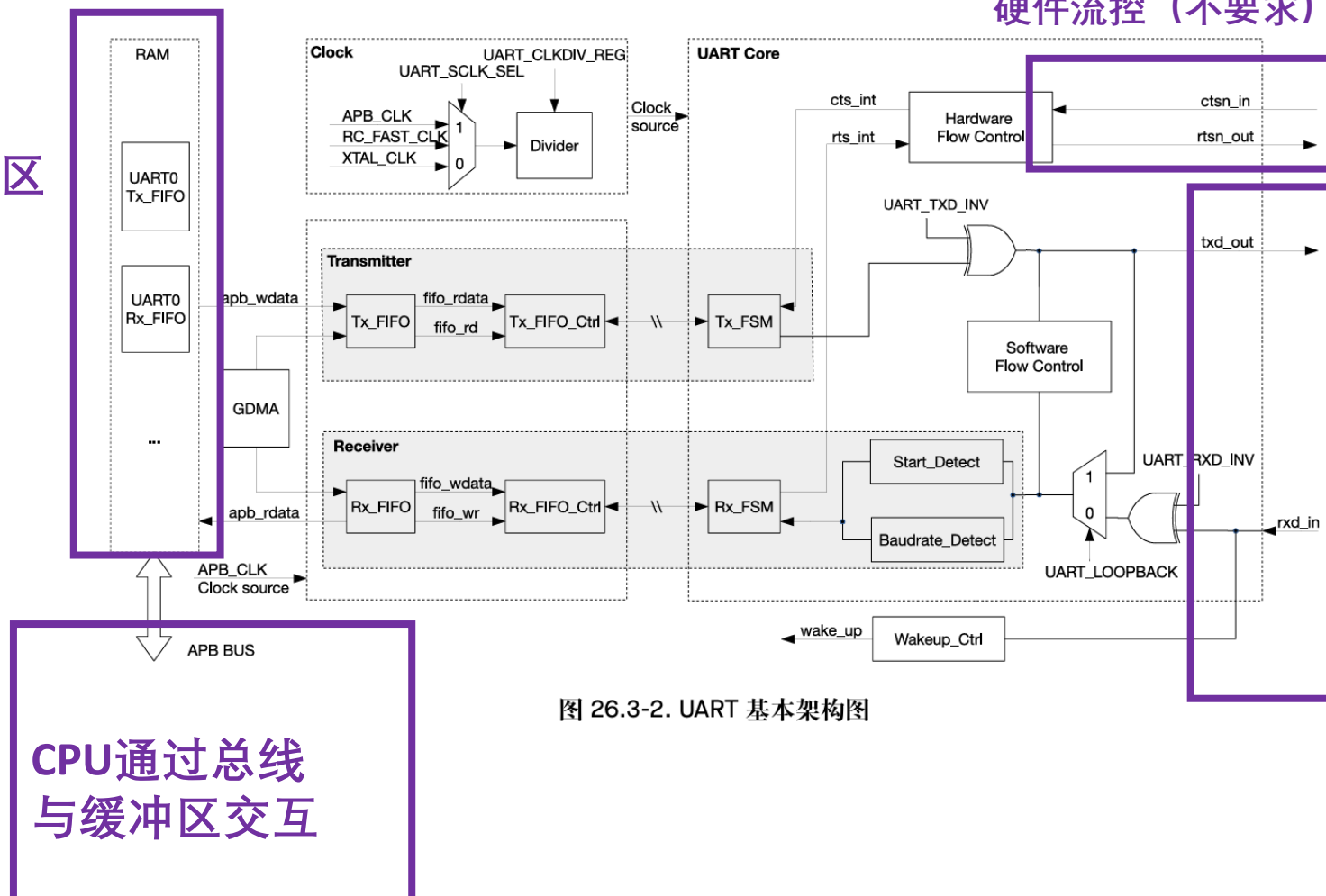


图 26.3-2. UART 基本架构图

## 2.ESP32-S3中的UART资源及其运用

- **硬件资源**：3个UART接口，分别命名为UART0、UART1、UART2，每个UART控制器都有一组独立的寄存器。可支持RS485协议。
- **板载CH340 (USB-TTL)**：默认绑定UART0，用烧录线即可实现UART连接。
- **硬件连接**：RX-TX TX-RX **GND-GND**，软件流控，无需接RTS、CTS。
- **代码实现**：无需外部库，使用Serial类，学会调用基本函数即可。注意波特率保持统一。

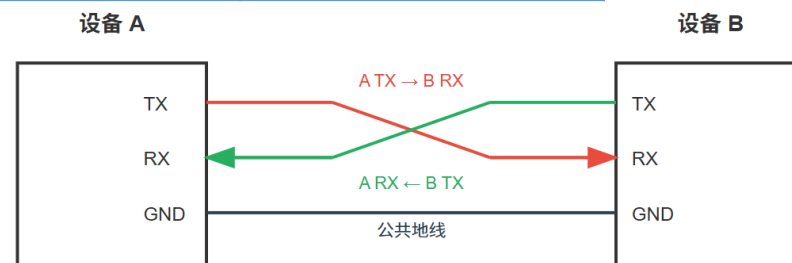
一定要保证共地！！

### ESP32-S3的UART硬件资源表

| 串 口   | RX固定管脚            | TX固定管脚 | 功能             |
|-------|-------------------|--------|----------------|
| UART0 | USB2 直连， 替代RX/TX  |        | 烧录、默认的<br>串口打印 |
|       | RX引脚              | TX引脚   |                |
| UART1 | GPIO18            | GPIO17 | 接外设， 不支<br>持烧录 |
| UART2 | 无固定， 指定空闲GPIO引脚即可 |        |                |

烧录时注意不要占用此串口

- **RX (Receive)**：此设备的接收
- **TX (Transmit)**：此设备的发送



## 2.ESP32-S3中的UART资源及其运用

- 硬件资源：3个UART接口，分别命名为UART0、UART1、UART2，每个UART控制器都有一组独立的寄存器。
- 板载CH340 (USB-TTL)：默认绑定UART0，用烧录线即可实现UART连接。
- 硬件连接：RX-TX TX-RX GND-GND，软件流控，无需接RTS、CTS。
- 代码实现：无需外部库，使用Serial类，学会调用基本函数即可。注意波特率保持统一。

### 实验一：ESP32通过USB连到PC，实现最基本的通信

实验目的：学习基本语句，理解UART的通信特点，应用到后续的硬件调试。

#### 学习基本语句：

一些参考：[通用异步接收器-发射器 \(UART\)](#) | [Arduino 文档](#)、[第6节 UART 通信](#) | [微雪文档平台](#)

//串口初始化函数，Serial<sub>n</sub>相当于已经申明好的对象

Serial.begin(115200); //默认，对应UART0

Serial1.begin(115200,18,17); //自定义，可以连接外设，参数：波特率、

串口格式（默认缺省，不写即可）、RX接收引脚、TX接收引脚

//串口打印，是单片机通过TX向外发送数据

Serial.print("Hello!");

Serial1.print(msg); //不换行，可以是字符串，也可以是变量名

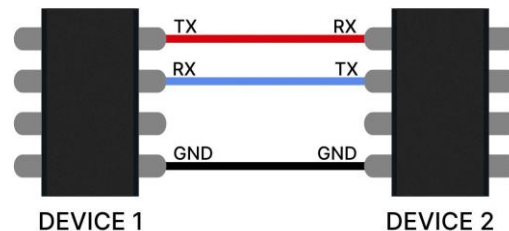
Serial.println("Hello!"); //换行

Serial.printf("Runtime = %lu ms\n",millis()); //格式化打印

//串口接收，是单片机通过RX接收数据

Serial.available(); //判断是否有新数据到来

Serial.read(); //读取，从串口缓冲区读取单个字符，返回char类型





## 2.ESP32-S3中的UART资源及其运用

- **硬件资源**: 3个UART接口, 分别命名为UART0、UART1、UART2, 每个UART控制器都有一组独立的寄存器。
- **板载CH340 (USB-TTL)**: 默认绑定UART0, 用烧录线即可实现UART连接。
- **硬件连接**: RX-TX TX-RX GND-GND, 软件流控, 无需接RTS、CTS。
- **代码实现**: 无需外部库, 使用Serial类, 学会调用基本函数即可。注意波特率保持统一。

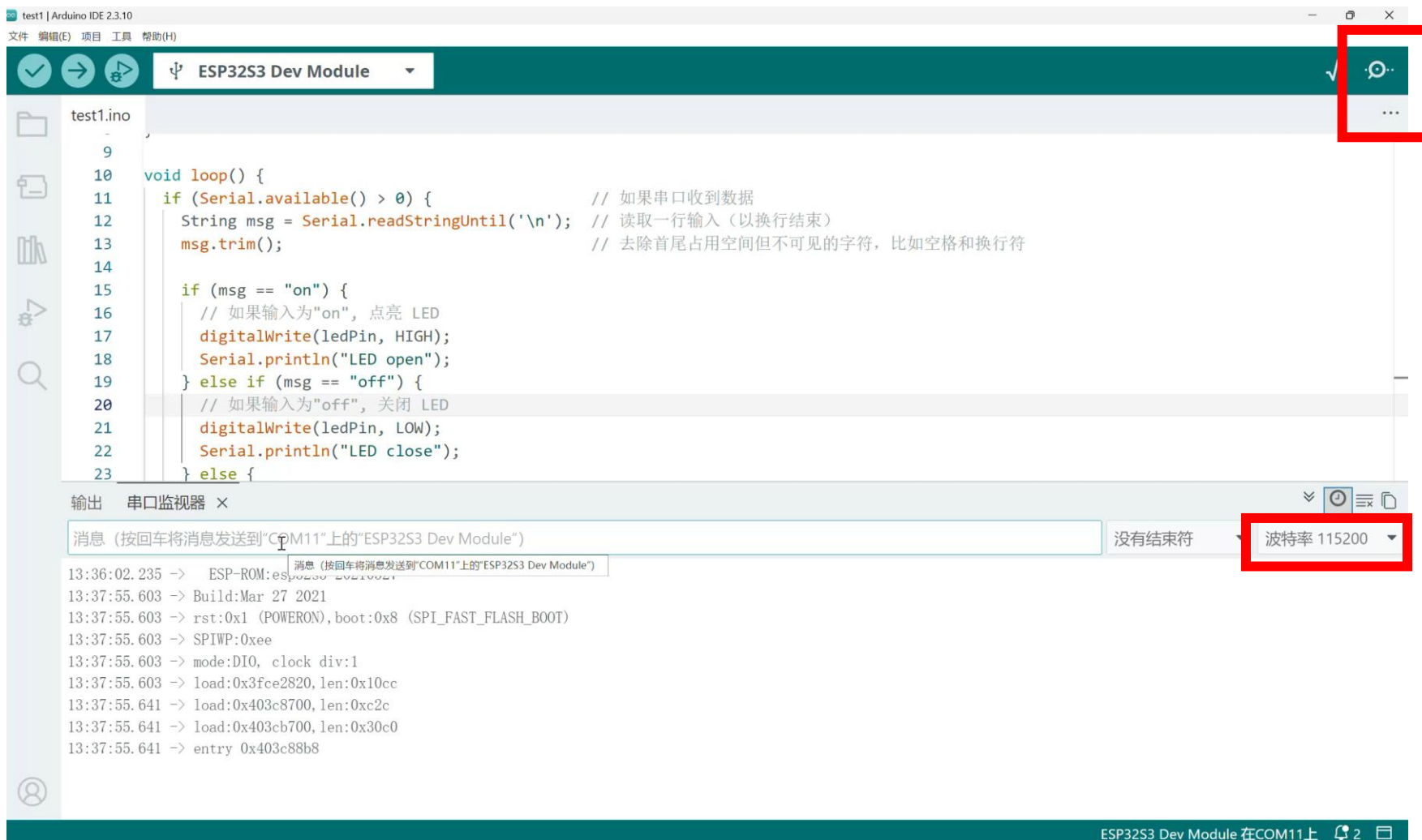
**实验一: ESP32通过USB连到PC, 实现最基本的通信, 用串口向单片机发送指令, 点亮LED。**

- **实验目的**: 学习基本语句, 理解UART的通信特点, 应用到后续的硬件调试。
- **硬件连接**: 直接用烧录线将ESP32和PC相连, 引脚7接LED。
- **代码如下**:

```
1 const int ledPin = 7; // 定义 LED 连接的引脚
2 void setup() {
3     pinMode(ledPin, OUTPUT); // 设置 LED 引脚为输出模式
4     Serial.begin(115200); // 初始化串口通信, 波特率 115200
5 }
```

```
6 void loop() {
7     if (Serial.available() > 0) { // 如果串口收到数据
8         // 读取一行输入 (以换行结束)
9         String msg = Serial.readStringUntil('\n');
10        // 去除首尾占用空间但不可见的字符, 比如空格和换行符
11        msg.trim();
12        if (msg == "on") {
13            // 如果输入为"on", 点亮 LED
14            digitalWrite(ledPin, HIGH);
15            Serial.println("LED open");
16        } else if (msg == "off") {
17            // 如果输入为"off", 关闭 LED
18            digitalWrite(ledPin, LOW);
19            Serial.println("LED close");
20        } else {
21            Serial.println("on or off"); // 提示有效输入
22        }
23    }
24 }
```

## 2.ESP32-S3中的UART资源及其运用-实验一演示视频



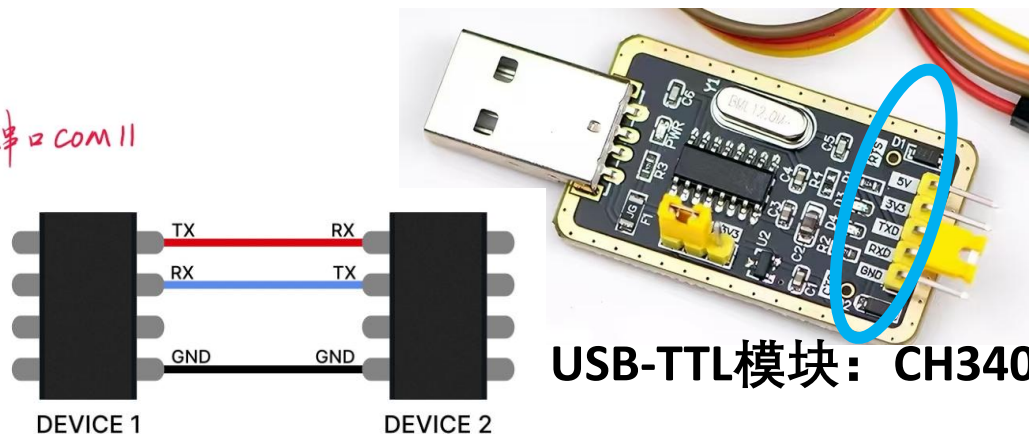
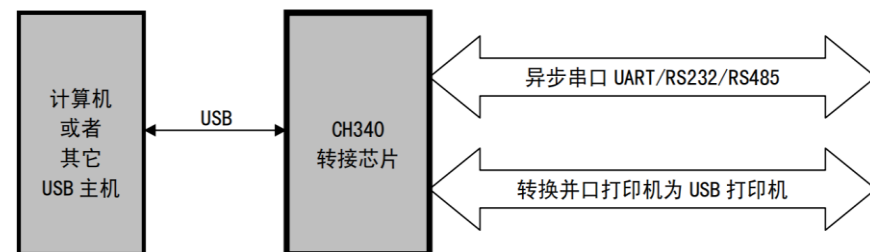
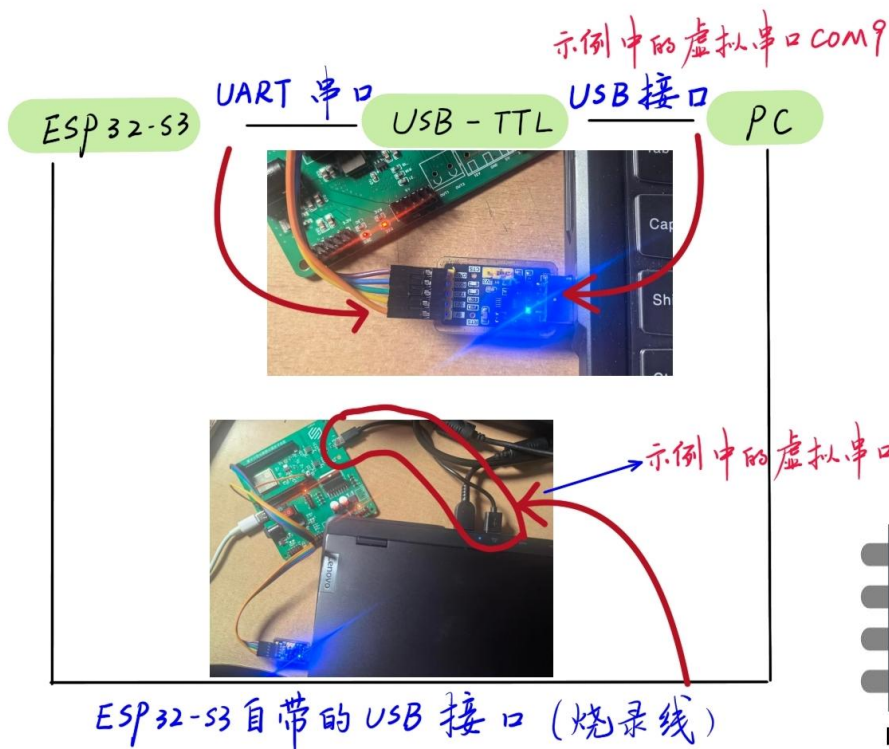
打开串口监视器，观察交互情况。如无响应，可首先尝试按下reset

## 2.ESP32-S3中的UART资源及其运用

- 硬件资源：3个UART接口，分别命名为UART0、UART1、UART2，每个UART控制器都有一组独立的寄存器。
- 板载CH340 (USB-TTL)：默认绑定UART0，用烧录线即可实现UART连接。
- 硬件连接：RX-TX TX-RX GND-GND，软件流控，无需接RTS、CTS。
- 代码实现：无需外部库，使用Serial类，学会调用基本函数即可。注意波特率保持统一。

**实验二：ESP32通过原装USB和USB-TTL连到PC，实现两端口的通信。**

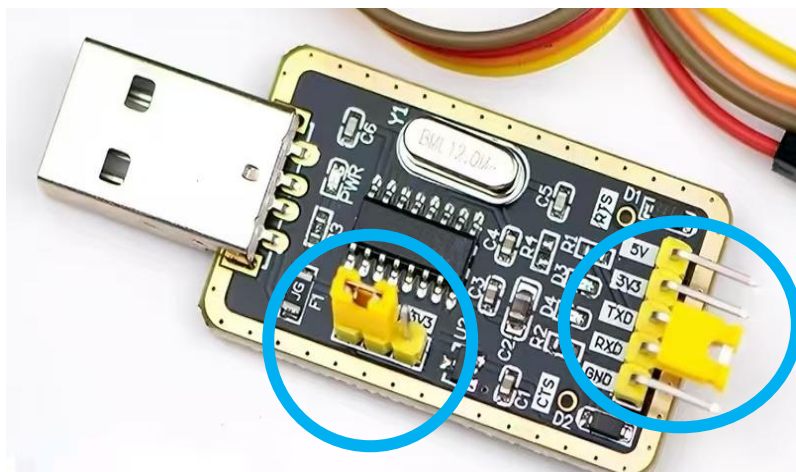
**实验目的：学习模块USB-TTL，学习使用串口调试助手，理解数据传输链路。**





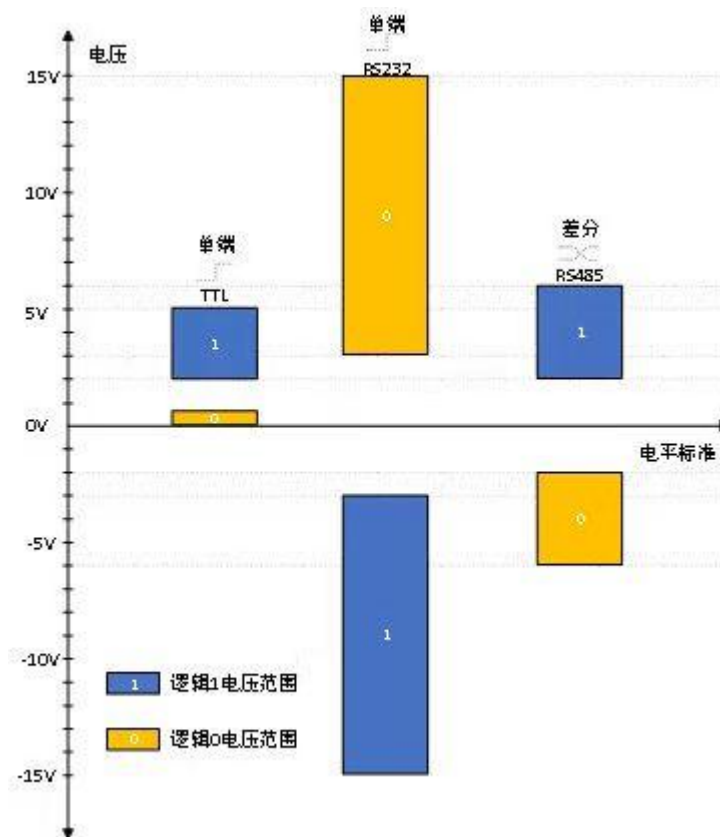
## 2.ESP32-S3中的UART资源及其运用

### 模块简介——USB-TTL (CH340)



跳帽的作用就是短接两个端子  
靠近USB口的跳帽不能拨开！  
作用：选择串口部分的供电（TTL输出）

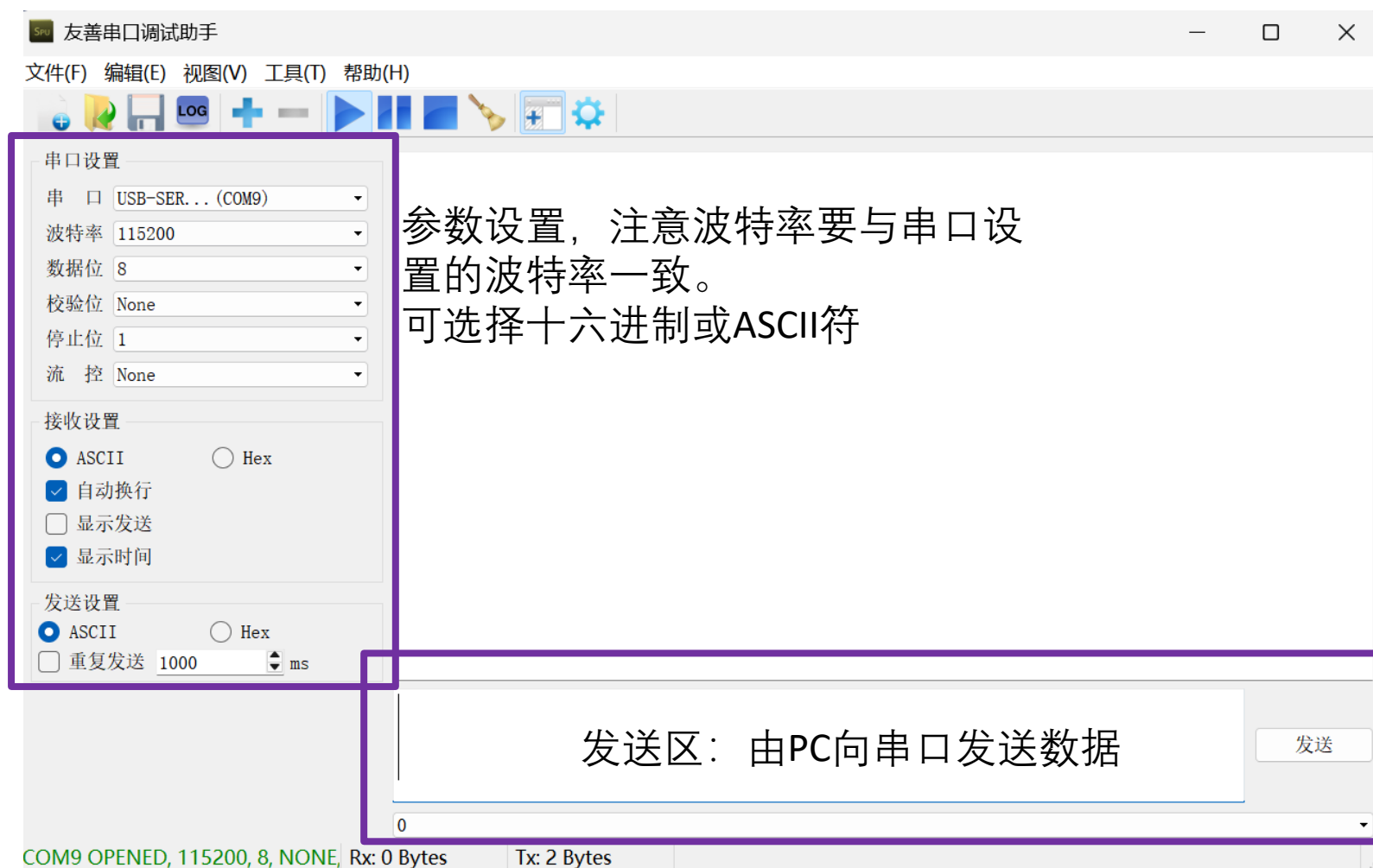
串口侧的跳帽要拔掉，一般不需要用  
此模块向外供电，所以5V/3.3V悬空即可。



几种协议的逻辑电平  
(TTL为左一)

## 2.ESP32-S3中的UART资源及其运用

### 工具讲解——串口调试助手



## 2.ESP32-S3中的UART资源及其运用

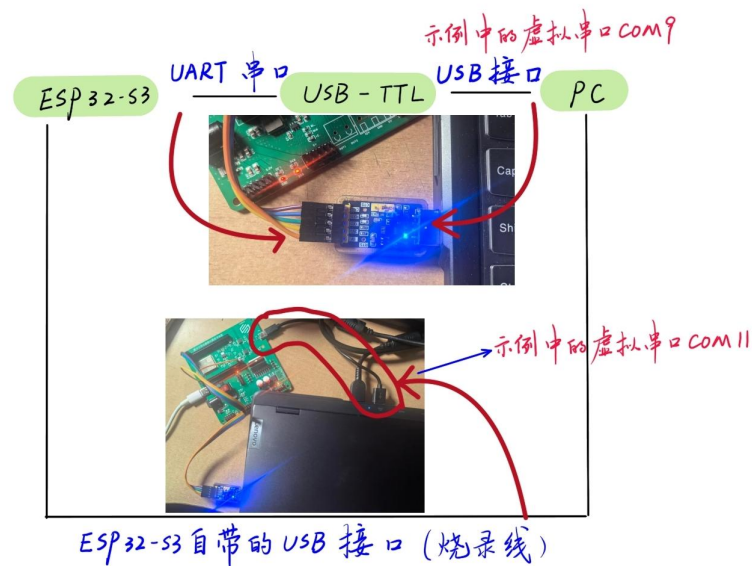
- 硬件资源：3个UART接口，分别命名为UART0、UART1、UART2，每个UART控制器都有一组独立的寄存器。
- 板载CH340 (USB-TTL)：默认绑定UART0，用烧录线即可实现UART连接。
- 硬件连接：RX-TX TX-RX GND-GND，软件流控，无需接RTS、CTS。
- 代码实现：无需外部库，使用Serial类，学会调用基本函数即可。注意波特率保持统一。

**实验二：ESP32通过原装USB和USB-TTL连到PC，实现两端口的通信。**

**实验目的：**学习模块USB-TTL，学习使用串口调试助手，理解数据传输链路。

**实验代码和硬件连接示意图：**

```
void setup() {  
    // put your setup code here, to run once:  
    Serial.begin(115200); //UART0, 串口监视器监测, COM11  
    Serial1.begin(115200, 18, 17); //UART1, 通过USB-TTL连到PC, COM9  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
    if(Serial1.available() > 0) { //如果单片机通过UART1收到了数据  
        char ch = Serial1.read(); //读取  
        Serial.print(ch); //单片机把该接收到的数据通过UART0发送  
    }  
}
```



## 2.ESP32-S3中的UART资源及其运用-实验二演示视频

The image shows a PC interface with two main windows. On the left is a '友善串口调试助手' (Friendly Serial Port Debug Assistant) window. It has a '串口设置' (Serial Port Settings) section where '串 口' (Serial Port) is set to 'USB-SER... (COM9)'. Below this are '接收设置' (Receive Settings) and '发送设置' (Send Settings). At the bottom, a text box contains 'Hello world!' and a '发送' (Send) button. The status bar at the bottom left says 'COM9 OPENED, 115200, 8, NONE, Rxc: 0 Bytes, Txc: 0 Bytes'. On the right is an 'Arduino IDE' window titled 'UART\_test1 | Arduino IDE 2.3.10'. The 'ESP32S3 Dev Module' is selected. The code in 'UART\_test1.ino' is as follows:

```
1 void setup() {  
2   // put your setup code here, to run once:  
3   Serial.begin(115200);  
4   Serial1.begin(115200, 18, 17);  
5 }  
6  
7 void loop() {  
8   // put your main code here, to run repeatedly:  
9   if(Serial1.available() > 0){  
10    char ch = Serial1.read();  
11    Serial.print(ch);  
12 }
```

Below the code editor is the '串口监视器' (Serial Monitor) window. It shows '输出' (Output) and '串口监视器' (Serial Monitor) tabs. The '消息' (Message) field contains '(按回车将消息发送到"COM11"上的"ES...'. The '波特率' (Baud Rate) is set to '115200'. A status bar at the bottom right says '行 6, 列 1 ESP32S3 Dev Module 在COM11上'.

串口为COM9, PC通过USBTTL向单片机发送信息

串口为COM11, 单片机通过数据线向PC发送信息, 显示在串口监视器

Hello world!

COM9 OPENED, 115200, 8, NONE, Rxc: 0 Bytes, Txc: 0 Bytes

行 6, 列 1 ESP32S3 Dev Module 在COM11上

## 2.ESP32-S3中的UART资源及其运用

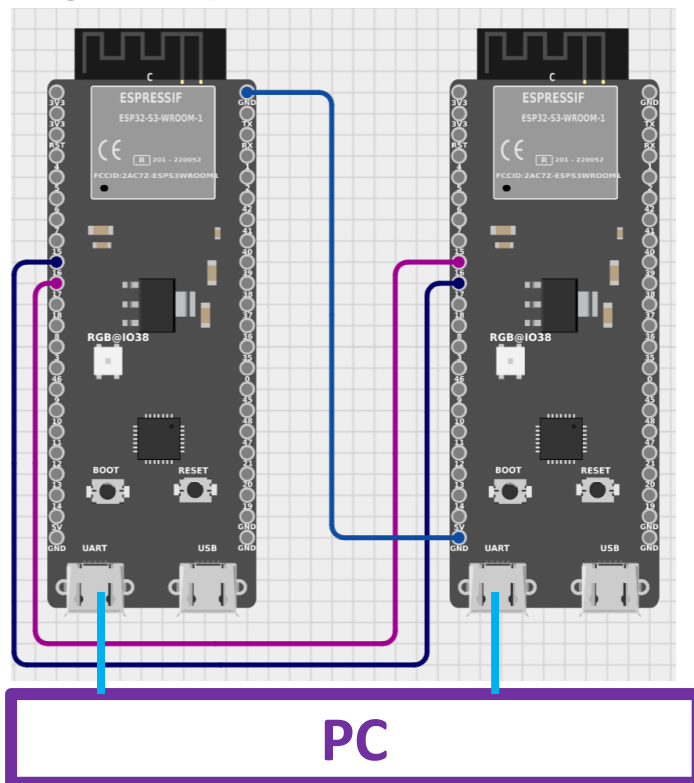
- **硬件资源**: 3个UART接口, 分别命名为UART0、UART1、UART2, 每个UART控制器都有一组独立的寄存器。
- **板载CH340 (USB-TTL)**: 默认绑定UART0, 用烧录线即可实现UART连接。
- **硬件连接**: RX-TX TX-RX GND-GND, 软件流控, 无需接RTS、CTS。
- **代码实现**: 无需外部库, 使用Serial类, 学会调用基本函数即可。注意波特率保持统一。

**实验三：两块ESP32通过UART相连，实现两板间通信。**

**实验目的：实现单片机间的通信。**

**硬件连接：**

A板RX—B板TX  
A板TX—B板RX  
两板GND相连



**代码实现：**

两块ESP32-S3, A板发送数据包,  
B板接收数据包。(仿照“数据帧”)

用各自的UART2实现板间通信,  
用各自的UART0实现与PC的通信:  
二者波特率可以不同(互不影响)

```
Serial.begin(115200);  
Serial2.begin(9600,SERIAL_8N1,16,17);
```

## 2.ESP32-S3中的UART资源及其运用

实验三：两块ESP32通过UART相连，实现两板间通信。

完整实验代码：

UART\_test3\_A.ino

```
1  int count = 0;
2  void setup() {
3      // put your setup code here, to run once:
4      Serial.begin(115200);
5      Serial2.begin(9600,SERIAL_8N1,16,17);
6  }
7
8  void loop() {
9      // put your main code here, to run repeatedly:
10     //发送数据包
11     String sendBuf = "MSG:" + String(count) + "|Hello ESP32-S3\r\n";
12     Serial2.print(sendBuf);
13     Serial.print("已发送: ");
14     Serial.println(sendBuf);
15
16     //接收应答
17     if(Serial2.available()>0){
18         String recvAck = Serial2.readStringUntil('\n');
19         Serial.print("收到应答: ");
20         Serial.println(recvAck);
21     }
22
23     count++;
24     delay(1000);
25 }
```

UART\_test3\_B.ino

```
1  void setup() {
2      // put your setup code here, to run once:
3      Serial.begin(115200);
4      Serial2.begin(9600, SERIAL_8N1, 16, 17);
5  }
6
7  void loop() {
8      // put your main code here, to run repeatedly:
9      // 检测是否有串口数据
10     if(Serial2.available() > 0){
11         String recvData = Serial2.readStringUntil('\n');
12         Serial.print("收到发送板数据: ");
13         Serial.println(recvData);
14
15         // 回复应答给发送板
16         String ack = "ACK_OK|" + recvData + "\r\n";
17         Serial2.print(ack);
18         Serial.print("回复应答: ");
19         Serial.println(ack);
20     }
21     delay(50);
22 }
```



## 2.ESP32-S3中的UART资源及其运用-实验三视频

The image displays two side-by-side screenshots of the Arduino IDE 2.3.10, showing the code and serial monitor for two different ESP32-S3 modules.

**Left Screenshot (UART\_test3\_A.ino):**

```
1 int count = 0;
2 void setup() {
3   // put your setup code here, to run once:
4   Serial.begin(115200);
5   Serial2.begin(9600, SERIAL_8N1, 16, 17);
6 }
7
8 void loop() {
9   // put your main code here, to run repeatedly:
10  String sendBuf = "MSG:" + String(count) + "|Hello ESP32-S3";
11  Serial2.print(sendBuf);
12  Serial.print("已发送: ");
13  Serial.println(sendBuf);
14
15  if(Serial2.available() > 0){
16    String rcvAck = Serial2.readStringUntil('\n');
17    Serial.print("收到应答: ");
18    Serial.println(rcvAck);
19  }
```

**Serial Monitor Output (Left):**

```
20:43:23.558 ->
20:43:24.535 -> 已发送: MSG:4|Hello ESP32-S3
20:43:24.573 ->
20:43:24.573 -> 收到应答: ACK_OK|MSG:3|Hello ESP32-S3
20:43:25.541 -> 已发送: MSG:5|Hello ESP32-S3
20:43:25.541 ->
20:43:25.541 -> 收到应答: ACK_OK|MSG:4|Hello ESP32-S3
```

**Right Screenshot (UART\_test3\_B.ino):**

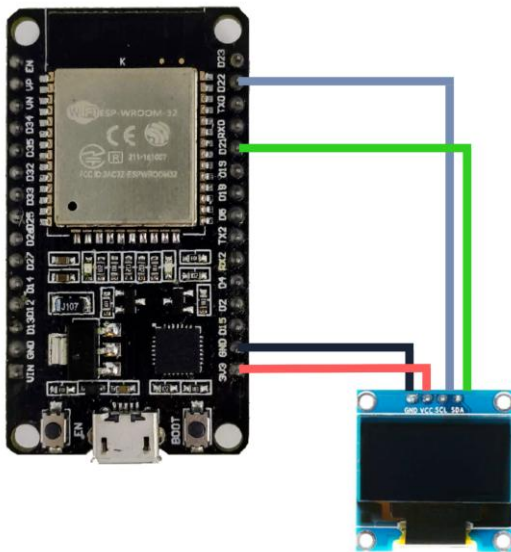
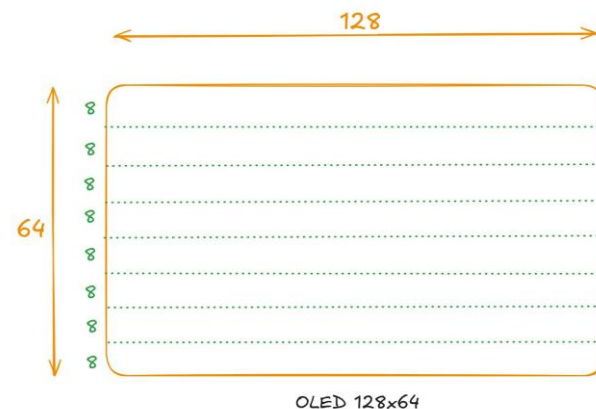
```
1 void setup() {
2   // put your setup code here, to run once:
3   Serial.begin(115200);
4   Serial2.begin(9600, SERIAL_8N1, 16, 17);
5 }
6
7 void loop() {
8   // put your main code here, to run repeatedly:
9   // 检测是否有串口数据
10  if(Serial2.available() > 0){
11    String rcvData = Serial2.readStringUntil('\n');
12    Serial.print("收到发送板数据: ");
13    Serial.println(rcvData);
14
15    // 回复应答给发送板
16    String ack = "ACK_OK|" + rcvData + "\r\n";
17    Serial2.print(ack);
18    Serial.print("回复应答: ");
19  }
```

**Serial Monitor Output (Right):**

```
20:43:23.594 -> 回复应答: ACK_OK|MSG:3|Hello ESP32-S3
20:43:23.594 ->
20:43:24.573 -> 收到发送板数据: MSG:4|Hello ESP32-S3
20:43:24.573 -> 回复应答: ACK_OK|MSG:4|Hello ESP32-S3
20:43:24.573 ->
20:43:25.585 -> 收到发送板数据: MSG:5|Hello ESP32-S3
20:43:25.585 -> 回复应答: ACK_OK|MSG:5|Hello ESP32-S3
20:43:25.585 ->
```

### 3. 示例模块——OLED (I2C通信协议)

- 单色屏：128×64个
- 硬件连接：I2C
- 阅读资料：[集成电路网](#)  
[\(I2C\) 协议](#) | [Arduino 文档](#)
- 代码实现：调用库函数  
U8g2, Arduino IDE 下载
- 一个取模工具：  
<https://led.baudance.com/>



硬件连接 (见左图)

VCC-3.3V (不能接到5V)

SCL-SCL (自行设定GPIO空闲引脚)

SDA-SDA (自行设定GPIO空闲引脚)

GND-GND

SCL(serial clock pin):

串行时钟引脚 (“同步”)

SDA(serial data pin):

串行数据引脚 (“总线”)

### 3.示例模块——OLED (I2C通信协议)

#### 学习基本语句:

##### //U8g2对象初始化

```
//初始化OLED (型号SSD1315/SSD1306都可,I2C地址0x3C, 128x64分辨率)
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /* reset=*/
U8X8_PIN_NONE,PIN_OLED_SCL, PIN_OLED_SDA);
//这个构造函数包含几个重要的参数:
//U8G2_SSD1306_128X64_NONAME_F_HW_I2C: 指定OLED型号和通信方式
//SSD1306: 驱动芯片型号
//128X64: 屏幕分辨率
//F: 全缓冲区模式 (占用更多内存但速度更快)
//HW_I2C: 使用硬件I2C
//U8G2_R0: 屏幕旋转角度 (R0=不旋转, R2=旋转180°)
//reset: 复位引脚 (使用U8X8_PIN_NONE表示不用)
//SCL、SDA引脚: 指定的空闲GPIO
```

##### //初始化设置和字体设置

```
u8g2.begin(); // 初始化显示屏
u8g2.enableUTF8Print(); // 启用UTF-8支持 (显示中文必需)
u8g2.setFont(u8g2_font_wqy12_t_chinese3); // 设置文泉驿中文字体(12x12)
u8g2.setFontDirection(0); // 设置文字方向 (0=正常)
```

##### //绘制屏幕内容

```
u8g2.firstPage();
do {
    u8g2.setCursor(0, 20); // 设置光标位置(x=0, y=20)
    u8g2.print("你好世界"); // 显示中文
    u8g2.setCursor(0, 45); // 设置光标位置(x=0, y=45)
    u8g2.print("Hello World"); // 显示英文
} while (u8g2.nextPage());
```

## 3.示例模块——OLED

//示例1 简单的静态显示

```
#include <Wire.h>
#include <U8g2lib.h>
```

// 初始化OLED

```
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0,U8X8_PIN_NONE,9,8);
```

```
void setup()
```

```
{
    u8g2.begin();                // 初始化显示屏
    u8g2.enableUTF8Print();      // 启用UTF-8支持 (可显示中文)
```

```
    u8g2.setFont(u8g2_font_wqy12_t_chinese3); // 设置文泉驿中文字体(12x12)
    u8g2.setFontDirection(0);                // 设置文字方向 (0=正常)
```

// 绘制屏幕内容

```
u8g2.firstPage();
```

```
do {
```

```
    u8g2.setCursor(0, 20);          // 设置光标位置(x=0, y=20)
    u8g2.print("你好世界");        // 显示中文
```

```
    u8g2.setCursor(0, 45);          // 设置光标位置(x=0, y=45)
    u8g2.print("Hello World");      // 显示英文
```

```
} while (u8g2.nextPage());
```

```
}
```

```
void loop()
```

```
{
    // 内容静态显示, 无需更新
}
```



### 3.示例模块——OLED

//示例2 简单的动态显示

```
#include <Wire.h>
```

```
#include <U8g2lib.h>
```

```
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE,9,8);
```

```
int counter = 0;
```

```
void setup()
```

```
{
```

```
    u8g2.begin();
```

```
    u8g2.enableUTF8Print();           // 启用UTF-8支持（显示中文）
```

```
}
```

```
void loop() {
```

```
    u8g2.firstPage();
```

```
    do {
```

```
        u8g2.setFont(u8g2_font_wqy12_t_chinese3);
```

```
        u8g2.setCursor(0, 20);
```

```
        u8g2.print("计数: ");
```

```
        u8g2.print(counter);
```

```
        u8g2.setCursor(0, 45);
```

```
        u8g2.print("Hello World");
```

```
    } while (u8g2.nextPage());
```

```
    counter++;
```

```
    delay(1000);
```

```
}
```



## 3.示例模块——OLED

### //示例3 非阻塞写法

// 步骤1: 引入头文件、定义硬件引脚、创建OLED对象

```
#include <Wire.h>
```

```
#include <U8g2lib.h>
```

// I2C引脚

```
constexpr uint8_t PIN_OLED_SDA = 8;
```

```
constexpr uint8_t PIN_OLED_SCL = 9;
```

// SSD1315屏幕I2C地址

```
constexpr uint8_t OLED_I2C_ADDRESS = 0x3C;
```

// 实例化128\*64 I2C OLED

```
U8G2_SSD1315_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE, PIN_OLED_SCL, PIN_OLED_SDA);
```

// 刷新定时全局变量

```
uint32_t lastDispMs = 0;
```

// 刷新间隔250毫秒

```
constexpr uint32_t DISP_INTERVAL = 250;
```

// 步骤2: 基础绘图函数

```
void drawMainScreen() {
```

```
    // 清空内存缓冲区
```

```
    oled.clearBuffer();
```

```
    // 设置字体
```

```
    oled.setFont(u8g2_font_6x13_tf);
```

```
    // 在坐标(10,20)绘制第一行文字
```

```
    oled.drawStr(10, 20, "ESP32-S3 SSD1315");
```

```
    // 在坐标(10, 40)绘制第二行文字
```

```
    oled.drawStr(10, 40, "I2C OLED Demo");
```

```
    // 将缓冲区数据一次性发送到屏幕
```

```
    oled.sendBuffer();
```

```
}
```





### 3.示例模块——OLED

// 步骤3: 工具函数+封装刷新调度函数

```
bool timeReached(uint32_t now, uint32_t deadline) {  
    return static_cast<int32_t>(now - deadline) >= 0;  
}  
void refreshScreen(uint32_t now) {  
    // 未到刷新时间直接退出  
    if (!timeReached(now, lastDispMs + DISP_INTERVAL)) {  
        return;  
    }  
    lastDispMs = now;  
    drawMainScreen();  
}
```

// 步骤4: 硬件初始化

```
void setup() {  
    Serial.begin(115200);  
    // 设置OLED I2C地址, 可选  
    oled.setI2CAddress(OLED_I2C_ADDRESS << 1);  
    // 执行SSD1306初始化命令  
    oled.begin();  
    Serial.println("OLED初始化完成");  
}
```

// 步骤5: 主循环持续调度刷新

```
void loop() {  
    uint32_t currentTime = millis();  
    refreshScreen(currentTime);  
}
```

## 02 编码器原理与应用



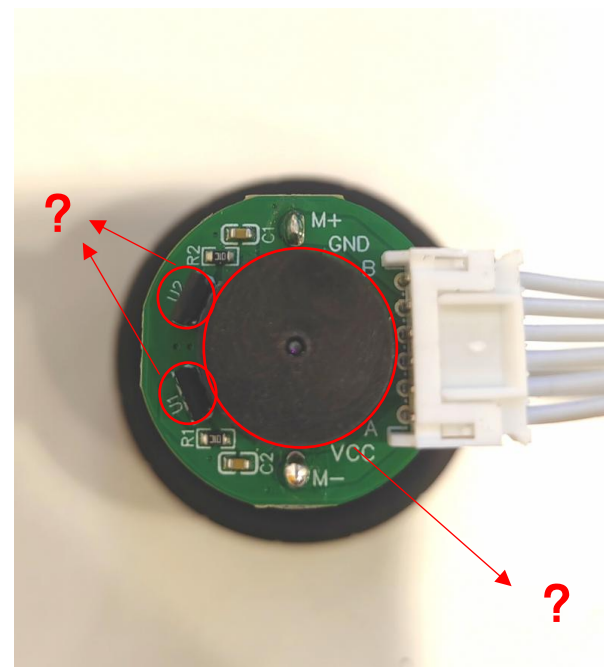
## 2.1 电机编码器

上节课大家学习了PWM波和电机驱动。

观察电机的尾部，会看到一个可以转动的圈圈和旁边两个米粒大小的东西。

这一套系统叫做“**编码器**”，用来实时检测电机的转速。

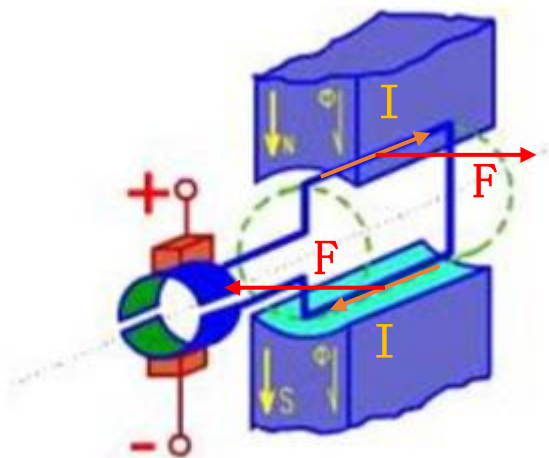
想要理解编码器的工作原理，我们首先要知道电机的工作原理。



## 2.1 电机编码器

想要理解编码器的工作原理，我们首先要知道电机的工作原理。

我们使用的电机，全称是“**直流减速电机**”，直流电机部分由电刷、换向片、绕组线圈、主磁极组成。



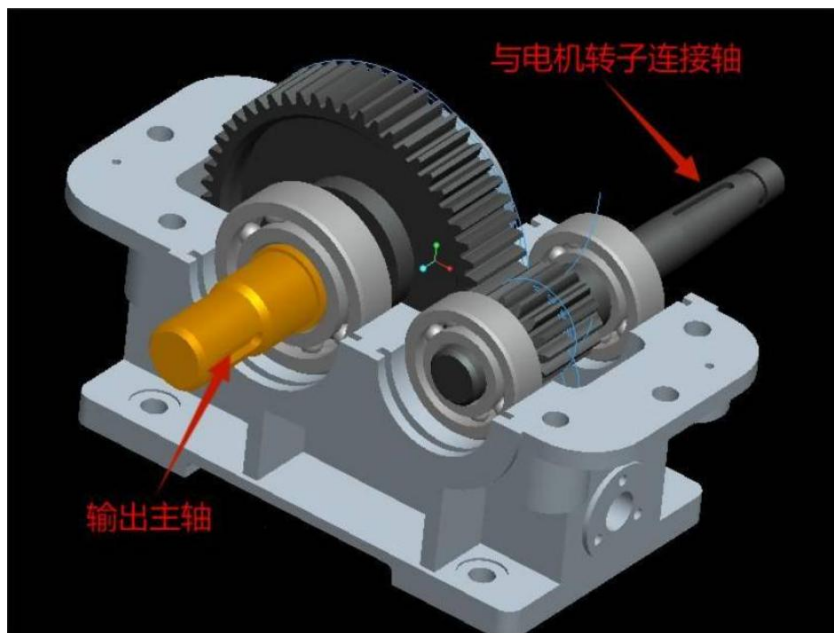
两个电刷之间通入直流电流，则绕组线圈中，靠近主磁极N极的导线，电流总是朝一个方向，靠近S极的导线，电流总是朝相反方向。

上下两段导线受到磁场的洛伦兹力方向相反，对线圈有相同的力矩，使得线圈总是沿着同一个方向转动。

因此，我们可以通过改变电流的方向来改变线圈的转动方向，通过改变直流电流的大小来改变线圈的转速。

## 2.1 电机编码器

为什么叫减速电机？这是因为直流电机的转子往往转速很高、转矩很小，无法有效的带动负载转动。



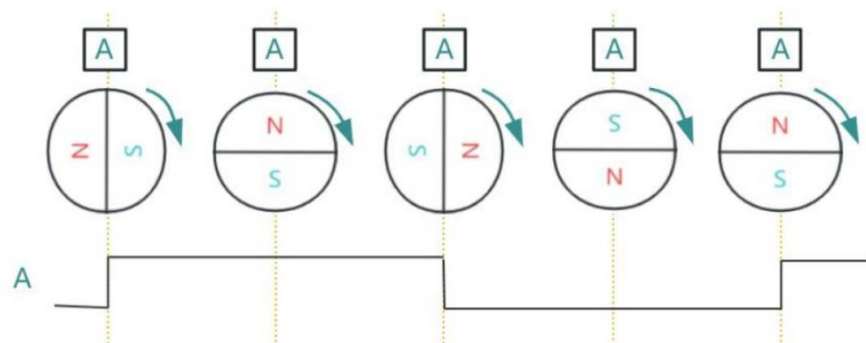
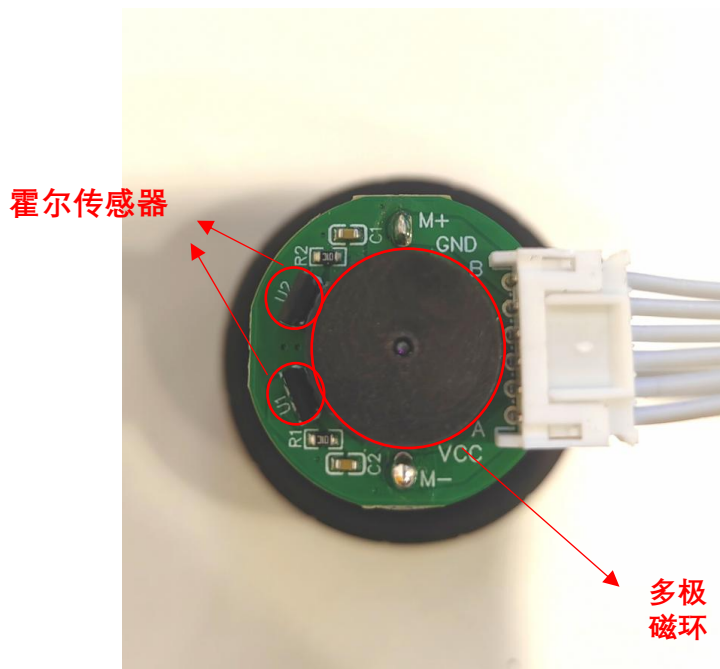
这个时候，我们将电机转子末端连接一个小齿轮，并通过一个大齿轮驱动输出主轴，就可以有效地降低转速、增大转矩。

连接电机转子的小齿轮，和连接输出主轴的大齿轮，二者的齿数之比，也就是输出转速和转子转速的比，我们称之为“**减速比**”。我们使用的电机减速比为  $1:30$ ，也就是说转子转30圈，输出主轴才会转一圈。

## 2.1 电机编码器

了解了直流减速电机的工作原理，我们再回头看编码器。

我们使用的电机自带的编码器，由一个**多极磁环**和两个**霍尔传感器A、B**组成。



假设磁环只有一个N-S对极，在磁环转动一圈的过程中，霍尔传感器A感应到磁场的改变，会输出一个周期的方波。

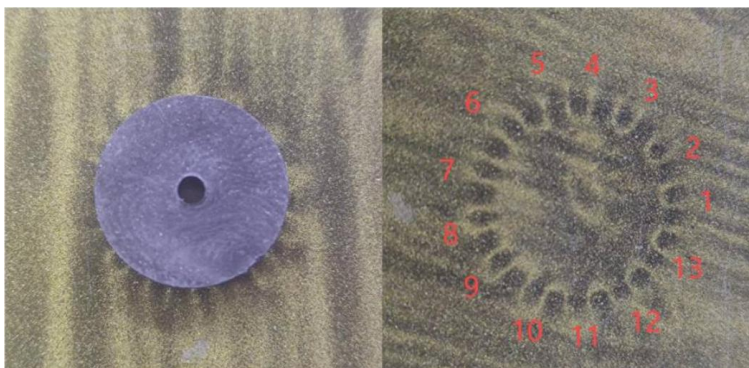
而实际上，多极磁环有N个N-S对极，在多极磁环转动一圈的过程中，霍尔传感器A会产生N个周期的方波，同理B也会产生N个周期的方波。



## 2.1 电机编码器

多极磁环有 $N$ 个N-S对极，在多极磁环转动一圈的过程中，霍尔传感器A会产生 $N$ 个周期的方波，同理B也会产生 $N$ 个周期的方波。

我们使用的多极磁环共有13个N-S对极。



我们使电机以某一转速转动，使用示波器观察一下A、B传感器的两相输出波形，会发现方波信号的频率约为2.2kHz，也就是每秒钟产生2.2k个周期的方波信号。

## 2.1 电机编码器

我们使电机以某一转速转动，使用示波器观察一下A、B传感器的两相输出波形，会发现方波信号的频率约为2.2kHz，也就是每秒钟产生2.2k个周期的方波信号。

我们已经知道，13个周期的方波信号对应磁环转一圈，而磁环连接电机转子，每转30圈对应输出主轴转一圈。

因此电机输出的转速为：

$$\frac{\text{方波频率}}{\text{磁环极数} \times \text{电机减速比}} = \frac{2200}{13 \times 30} = 5.64 \text{ rps}$$

也就是338 rpm。



我们再观察两相输出波形，会发现有一个 $\frac{\pi}{2}$ 左右的相位差。我们假设当前的转动方向为正，那么可以发现正向转动的特征为：

**A的上升沿对应B的高电平，**

**A的下降沿对应B的低电平。**

显然，当电机反向转动时，A和B的输出均反相，那么反向转动的特征为：

**A的上升沿对应B的低电平，**

**A的下降沿对应B的高电平。**

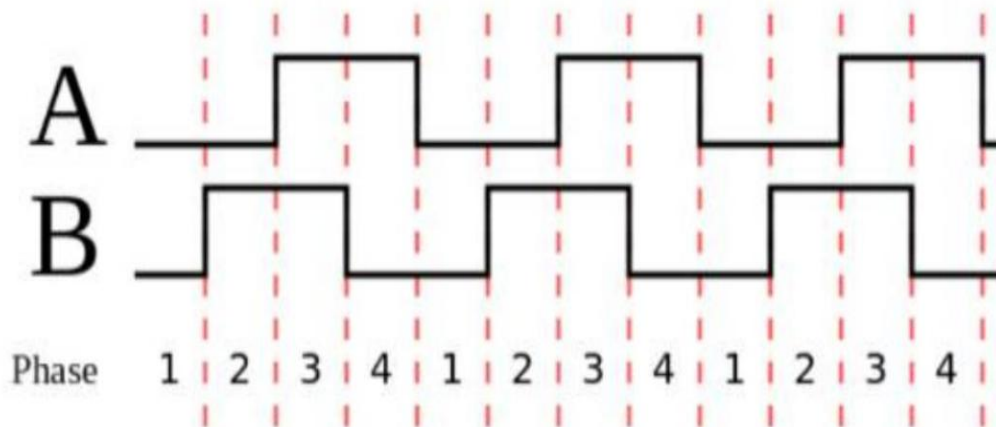
## 2.2 四倍频法计算转速

那么经过以上讨论，我们就可以总结出利用编码器获取电机转向和转速的方案：

计算A相的方波频率，除以磁环极数和减速比，获得转速大小；  
查看A相边沿（上升沿/下降沿）时B相的高低电平，获得转向。

这种计算方法，工程上称之为**单边沿法**。

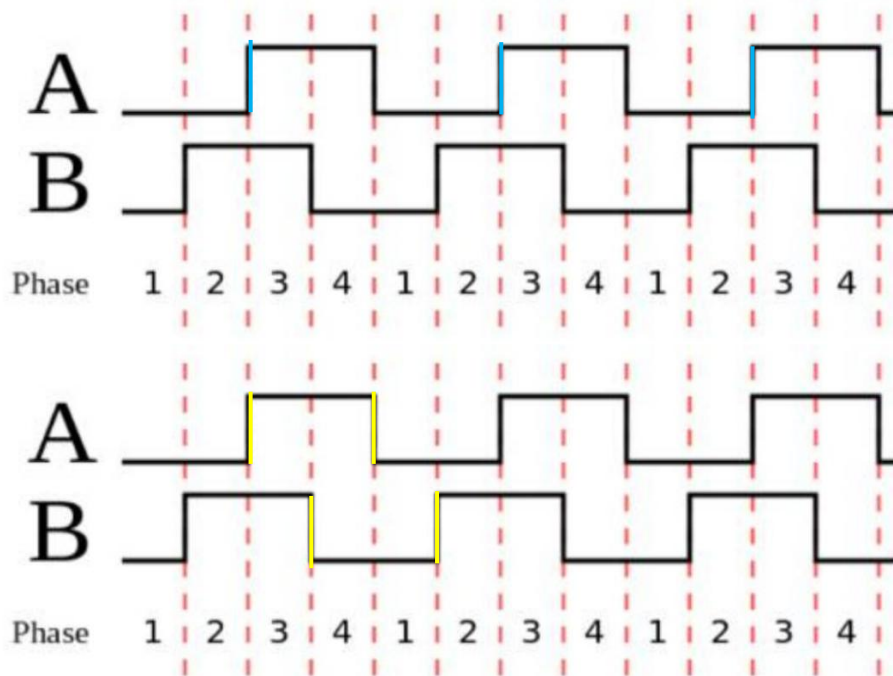
为了提高转速计算的精确度，我们介绍另一种更常用的计算方法：**四倍频法**。



## 2.2 四倍频法计算转速

为了提高转速计算的精确度，我们介绍工程上常用的一种计算方法：**四倍频法**。

我们利用A、B两相输出的所有边沿，根据另一相的电平高低进行计数。这样，在原来A相的一个周期内，就进行了四次计数，有效地提高了计数的精确度。



磁环极数PPR，  
电机减速比gear\_ratio，  
取一小段时间间隔interval，  
该段时间间隔内，边沿计数counts，  
则电机转速

$$= \frac{\text{counts}}{\text{interval} \times \text{PPR} \times \text{gear\_ratio} \times 4} \times 60 \text{ (rpm)}$$

编程实现？中断！

# 回顾：ESP32-S3的中断机制

外部中断触发类型：

1. **DISABLED**：关闭中断
2. **RISING**：上升沿触发
3. **FALLING**：下降沿触发
4. **CHANGE**：边沿触发，上升沿和下降沿均可
5. **ONLOW**：低电平触发，处理器响应前需要保持电平，否则会丢失中断
6. **ONHIGH**：高电平触发，处理器响应前需要保持电平
7. **ONLOW\_WE**：低电平触发，同时将处理器从休眠模式中唤醒，处理器响应前需要保持电平
8. **ONHIGH\_WE**：高电平触发，同时将处理器从休眠模式中唤醒，处理器响应前需要保持电平

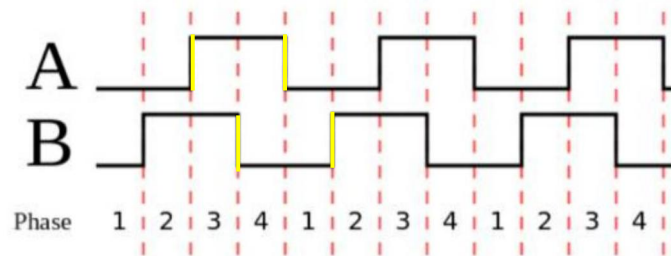
## 2.2 四倍频法计算转速

我们使用attachInterrupt函数，将A、B两相输出的边沿与对应的中断处理函数绑定，就可以在边沿到来时自动完成计数。

```
void ARDUINO_ISR_ATTR encodeA() {
    if (digitalRead(ENC_A) == digitalRead(ENC_B)) {
        encoderCount++; // A上升沿B高电平，或A下降沿B低电平，正向转动
    } else {
        encoderCount--; // A上升沿B低电平，或A下降沿B高电平，反向转动
    }
} // A相边沿中断处理函数

void ARDUINO_ISR_ATTR encodeB() {
    if (digitalRead(ENC_A) == digitalRead(ENC_B)) {
        encoderCount--; // B上升沿A高电平，或B下降沿A低电平，反向转动
    } else {
        encoderCount++; // B上升沿A低电平，或B下降沿A高电平，正向转动
    }
} // B相边沿中断处理函数
```

```
void setup() {
    Serial.begin(115200);
    // 上拉输入
    pinMode(ENC_A, INPUT_PULLUP);
    pinMode(ENC_B, INPUT_PULLUP);
    // 中断处理函数绑定
    attachInterrupt(digitalPinToInterrupt(ENC_A), encodeA, CHANGE);
    attachInterrupt(digitalPinToInterrupt(ENC_B), encodeB, CHANGE);
}
```



**attachInterrupt** The function attachInterrupt is used to attach the interrupt to the defined pin.

```
attachInterrupt(uint8_t pin, voidFuncPtr handler, int mode);
```

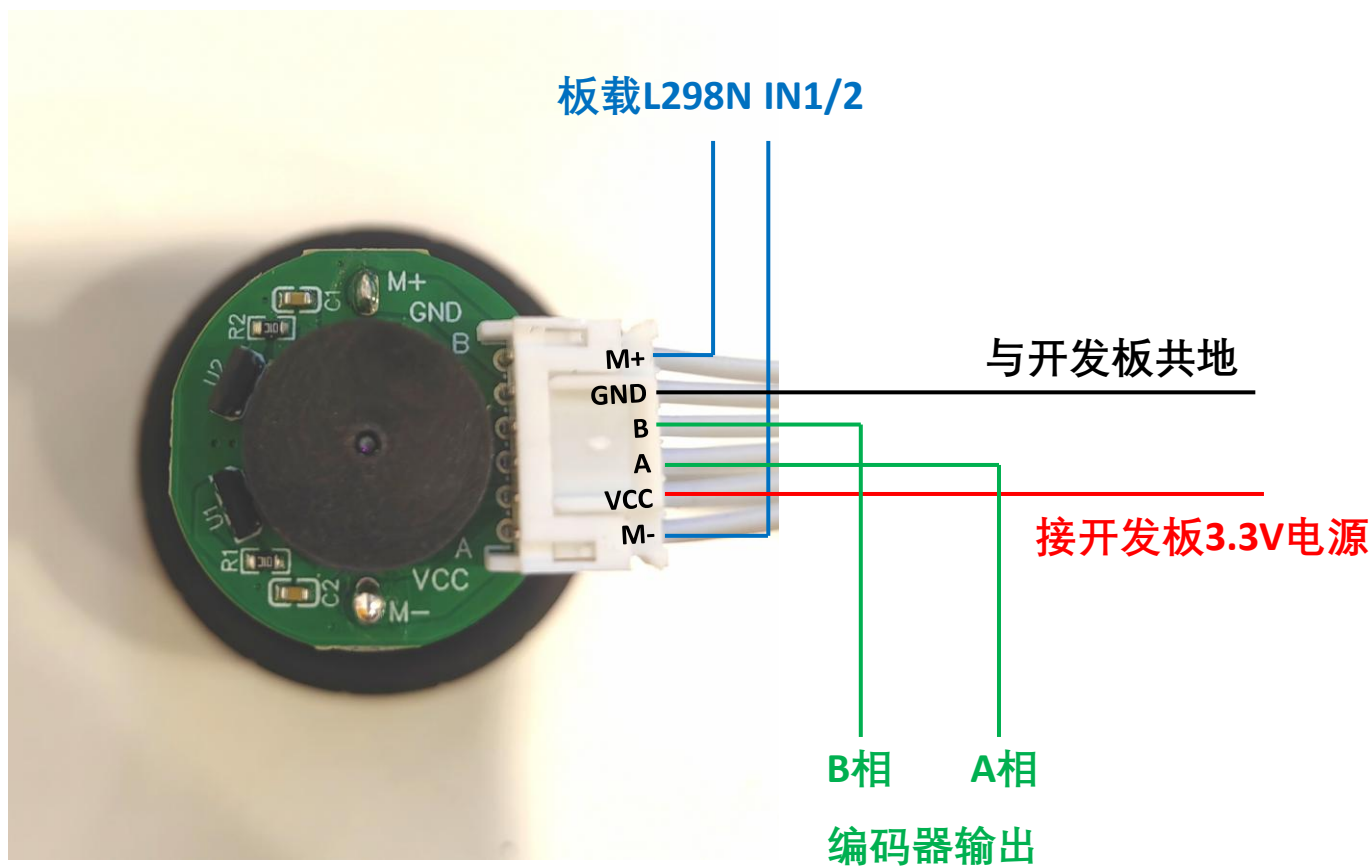
- pin defines the GPIO pin number.
- handler set the handler function.
- mode set the interrupt mode.



## 2.2 四倍频法计算转速

编码器如何接线？

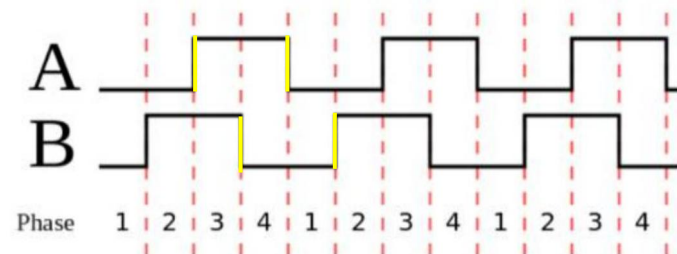
板载L298N IN1/2



## 2.2 四倍频法计算转速

四倍频算法比较固定，能否对代码进行封装复用？

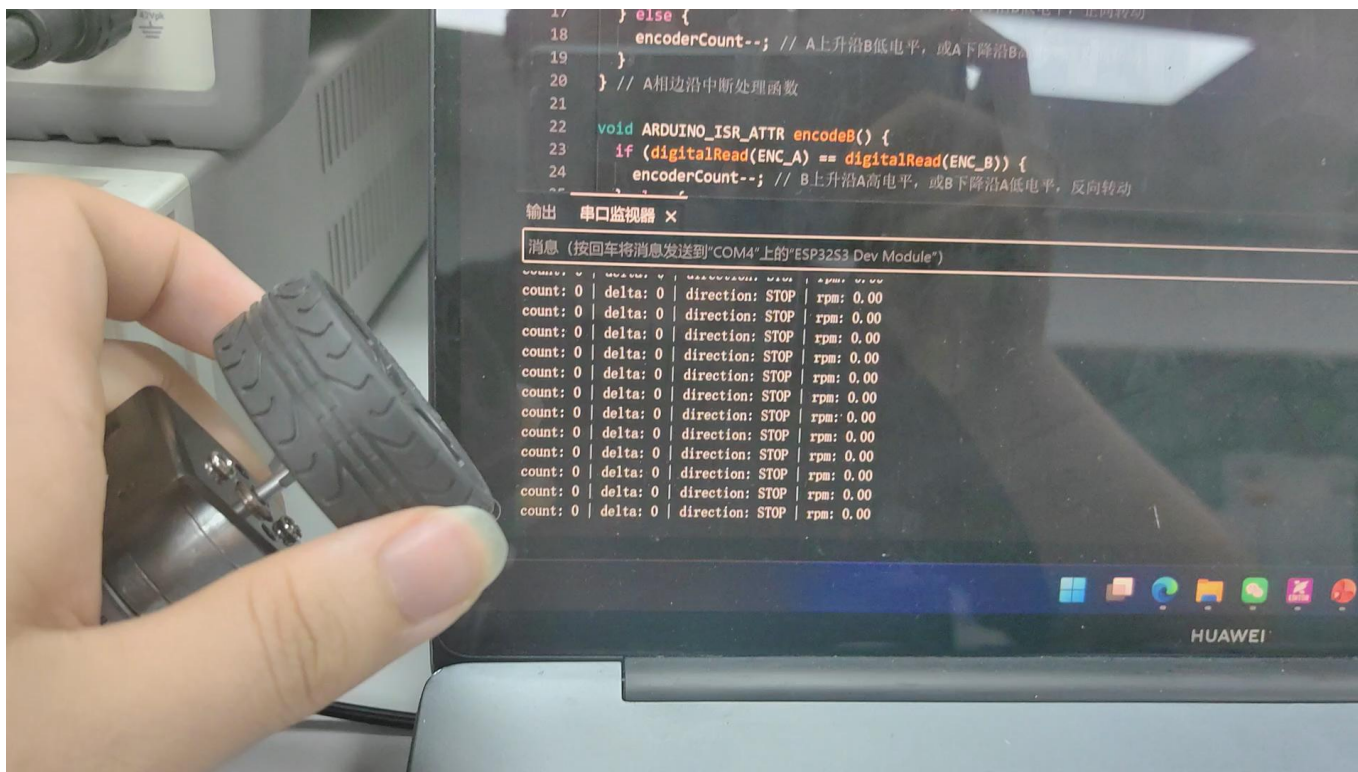
**ESP32Encoder.h**封装了中断处理函数，只需绑定A/B引脚，调用getCount()函数即可获得计数结果。



```
// 构造函数和析构函数
ESP32Encoder(bool always_interrupt=false, enc_isr_cb_t enc_isr_cb=nullptr, void* enc_isr_cb_data=nullptr);
~ESP32Encoder();

// 三种计算方法的绑定引脚函数
void attachHalfQuad(int aPinNumber, int bPinNumber); // 只计算一部分的边沿，“双倍频”
void attachFullQuad(int aPinNumber, int bPinNumber); // 四倍频，计算两相所有边沿
void attachSingleEdge(int aPinNumber, int bPinNumber); // 单边沿，只计算某一相的某一种沿

// 计数函数
int64_t getCount(); // 读取当前累计计数值
int64_t clearCount(); // 清空计数值
int64_t pauseCount(); // 暂停计数
int64_t resumeCount(); // 恢复计数
void detach(); // 解除引脚绑定，释放硬件资源
```

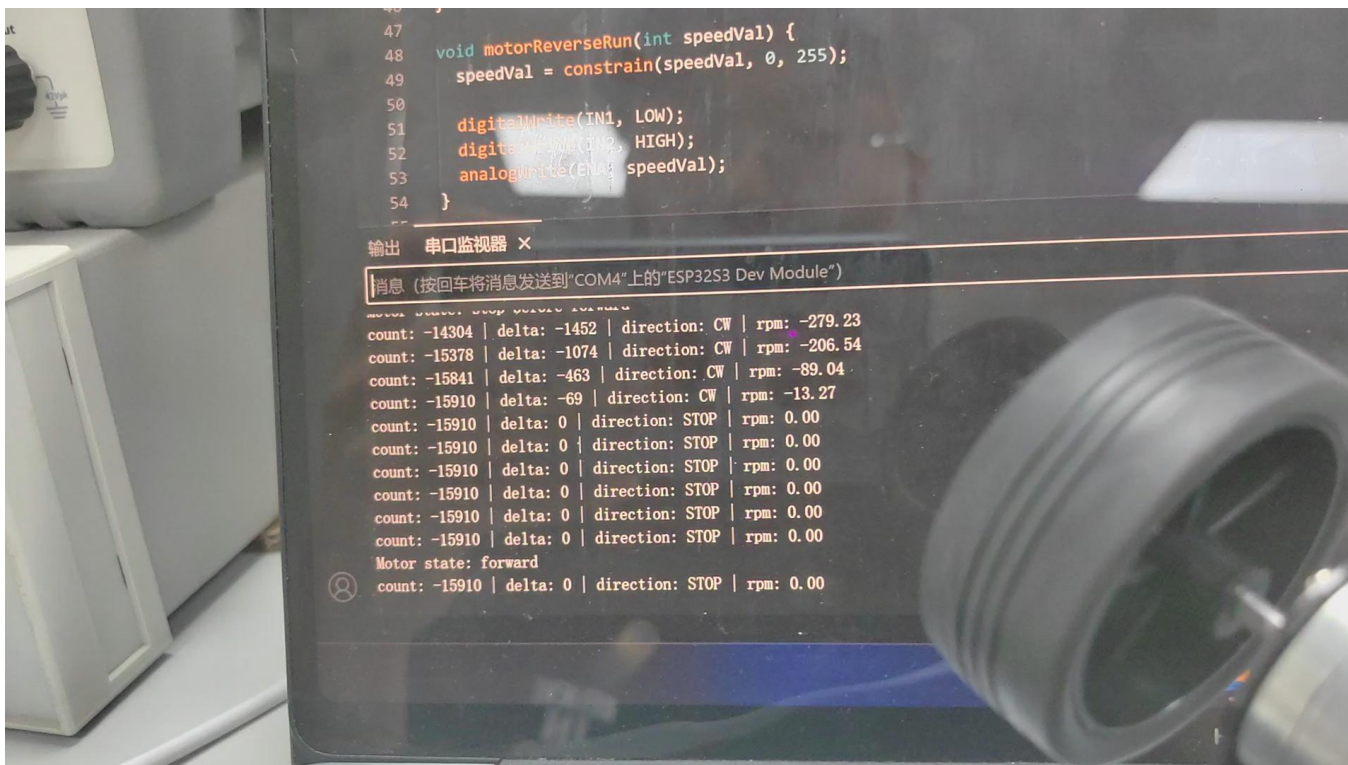
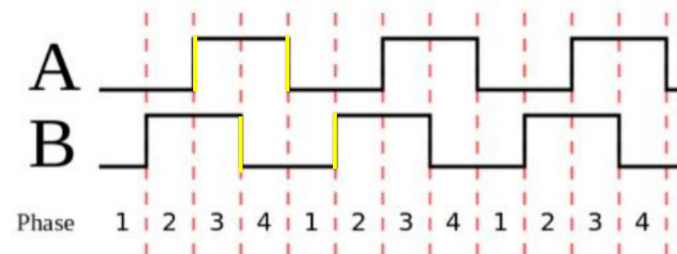


## 2.2 四倍频法计算转速

实战练习：

用手拨动电机，并实时测速。

使用L298N驱动电机转动，并实时测速。

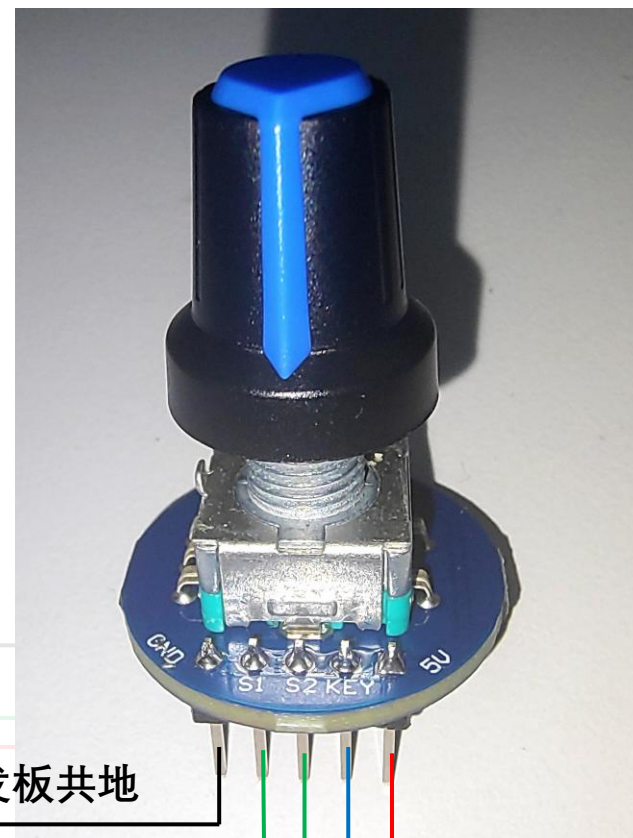
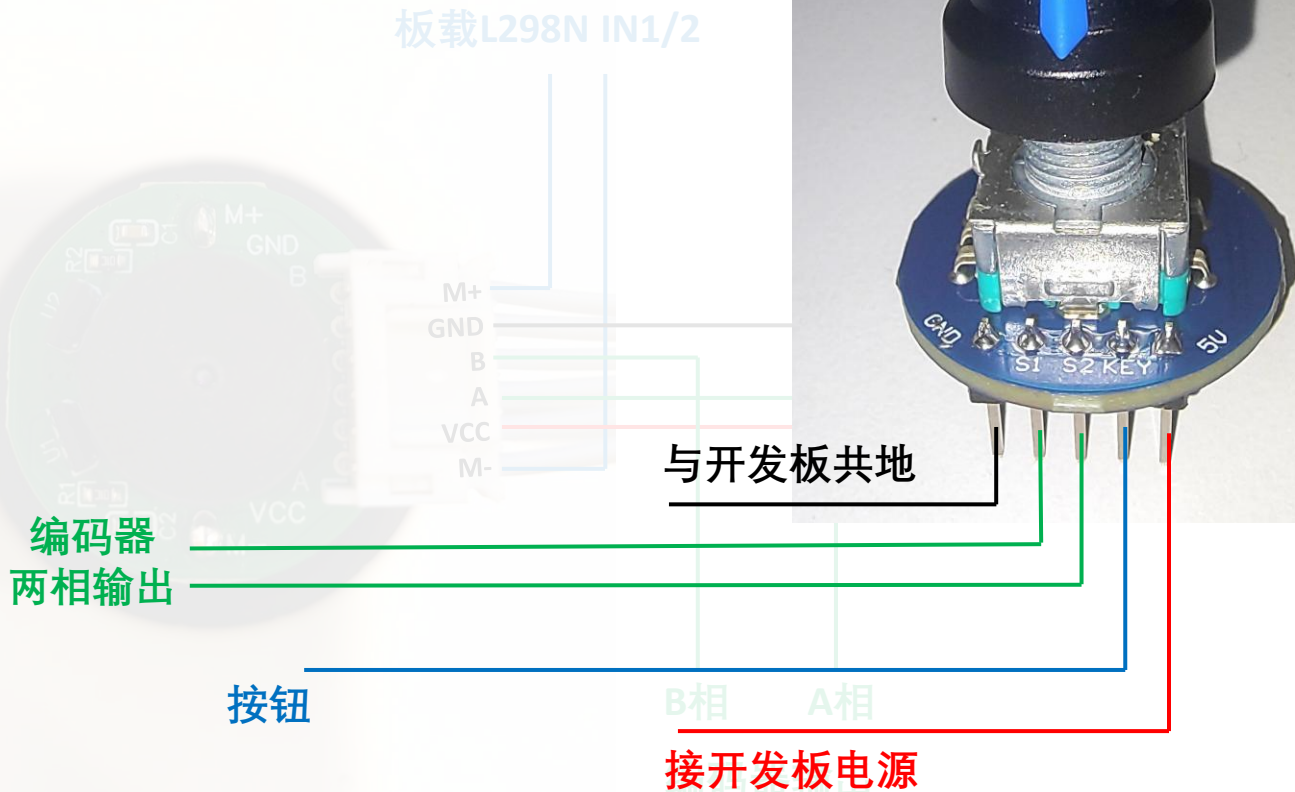




## 2.3 旋转编码器

电机编码器 VS 旋转编码器，有何区别？

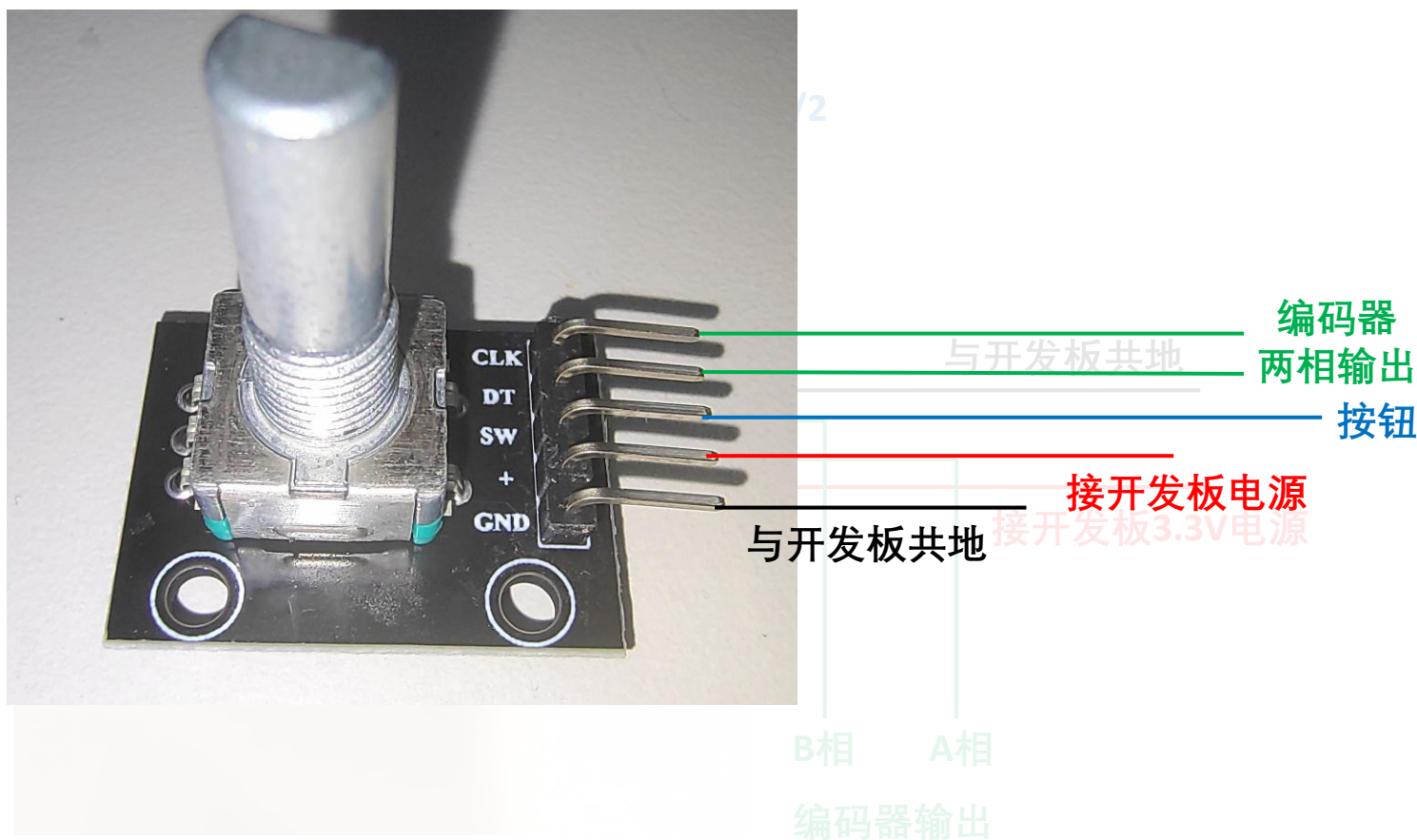
编码器如何接线？



## 2.3 旋转编码器

电机编码器 VS 旋转编码器，有何区别？

编码器如何接线？

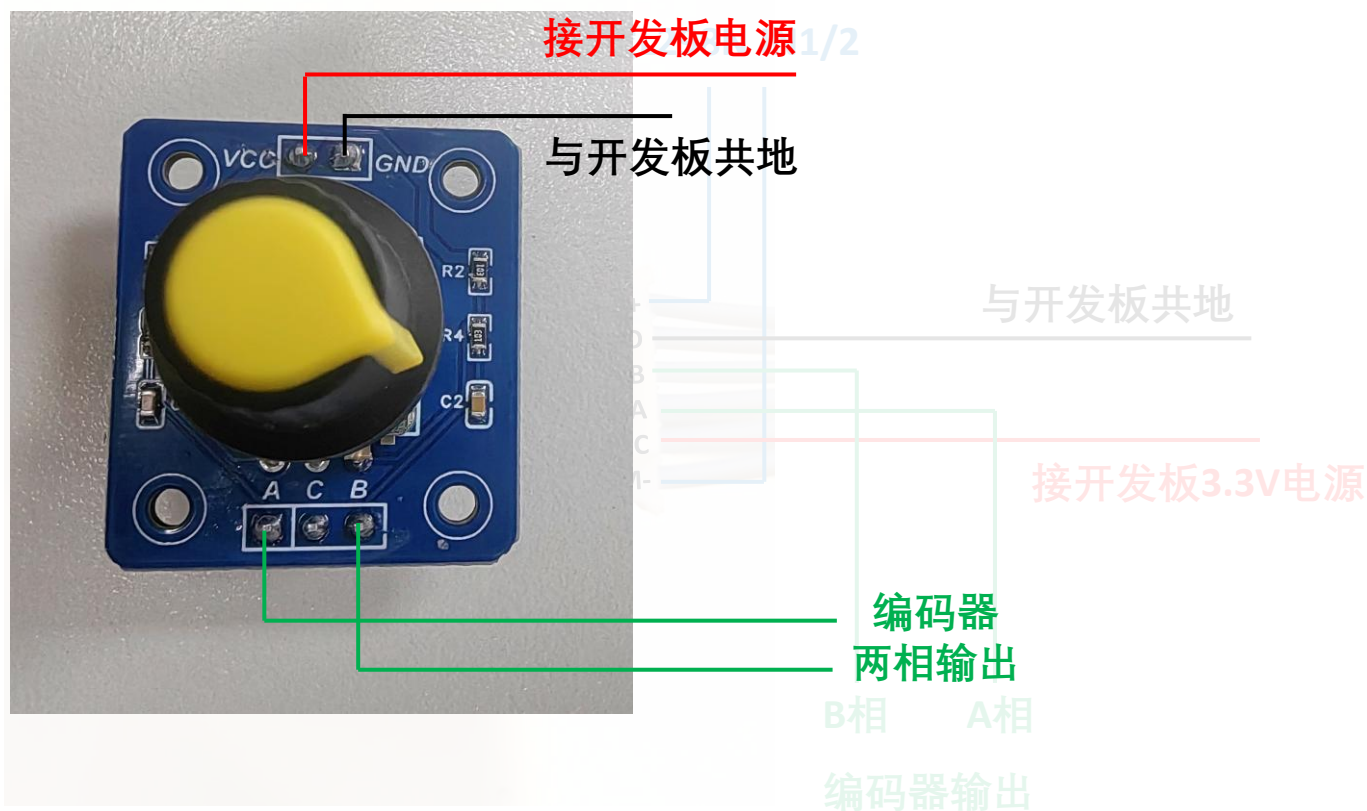




## 2.3 旋转编码器

电机编码器 VS 旋转编码器，有何区别？

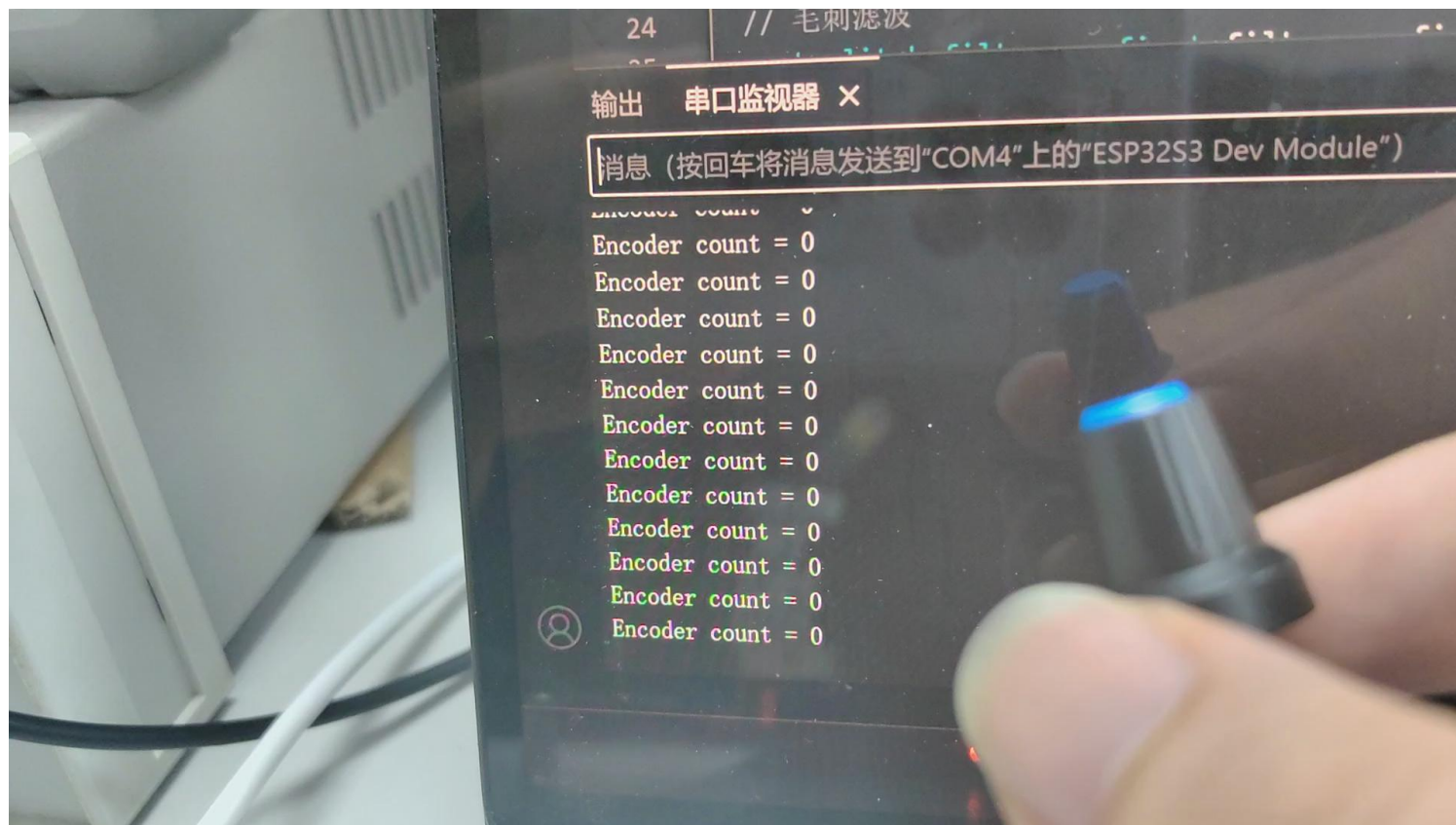
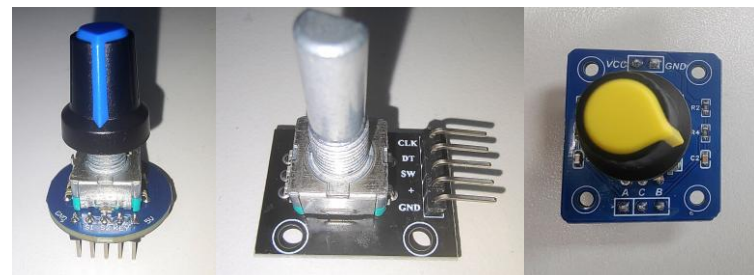
编码器如何接线？



## 2.3 旋转编码器

实战练习：

用手拧动旋转编码器，并使用中断进行计数。



## 2.3 旋转编码器

电机编码器 VS 旋转编码器，有何区别？

编码器如何接线？

|         | 电机编码器                       | 旋转编码器                     |
|---------|-----------------------------|---------------------------|
| 主要用途    | 测量电机转速和方向，<br>以控制和调节电机的运动状态 | 测量编码器当前位置，<br>用于参数调节，菜单选择 |
| 输入方式    | 电机自动旋转                      | 人手拧动                      |
| 输出信号    | A/B相方波                      | A/B相脉冲，部分有按钮              |
| 计数值处理   | 更关注计数值随单位时间的变化，计算转速         | 更关注计数值本身的大小，计算位置          |
| 分辨率     | 较高，每圈390个脉冲                 | 较低，每圈20个脉冲                |
| 转速      | 可以非常快                       | 非常慢                       |
| 对实时性的要求 | 要求较高，漏数脉冲会影响转速计算结果          | 要求较低，抖动可以通过软件处理           |
| 常用计数方案  | GPIO中断，？                    | GPIO中断，？                  |

B相 A相

编码器输出

## 2.4 利用PCNT进行编码器计数

$$\text{电机转速} = \frac{\text{counts}}{\text{interval} \times PPR \times \text{gear\_ratio} \times 4} \times 60 \text{ (rpm)}$$

编程实现？中断？

问题：当电机转速很快的时候，脉冲频率过高，CPU受到**频繁的中断**，影响其他程序的正常执行。

思考：需要一个**自动进行边沿计数**的硬件机制，CPU只需要定时读取计数结果就好。

解决方案：ESP32S3的**脉冲计数控制器（PCNT）**

## 2.4 利用PCNT进行编码器计数

### 解决方案：ESP32S3的脉冲计数控制器（PCNT）

PCNT 有如下特性：

- 四个脉冲计数控制器（单元），各自独立工作，计数范围是 1 ~ 65535
- 每个单元有两个独立的通道，共用一个脉冲计数控制器
- 所有通道均有输入脉冲信号（如 sig\_ch0\_un）和相应的控制信号（如 ctrl\_ch0\_un）
- 滤波器独立工作，过滤每个单元输入脉冲信号（sig\_ch0\_un 和 sig\_ch1\_un）控制信号（ctrl\_ch0\_un 和 ctrl\_ch1\_un）的毛刺
- 每个通道参数如下：
  1. 选择在输入脉冲信号的上升沿或下降沿计数
  2. 在控制信号为高电平或低电平时可将计数模式配置为递增、递减或停止计数
- 最大脉冲频率：40 MHz

## 2.4 利用PCNT进行编码器计数

解决方案：ESP32S3的脉冲计数控制器（PCNT）

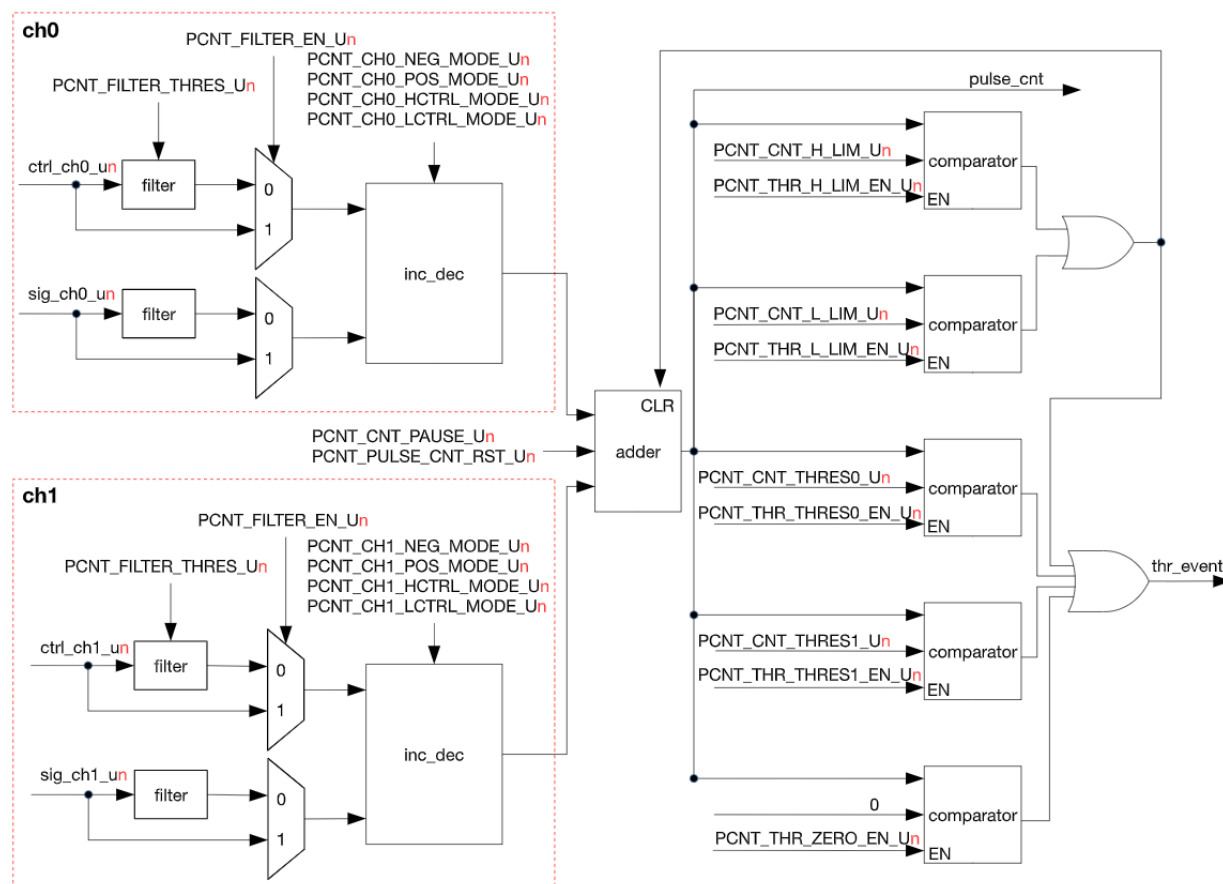


图 38.2-1. PCNT 单元基本架构图



## 2.4 利用PCNT进行编码器计数

实战练习：

使用PCNT，对旋转编码器进行计数。

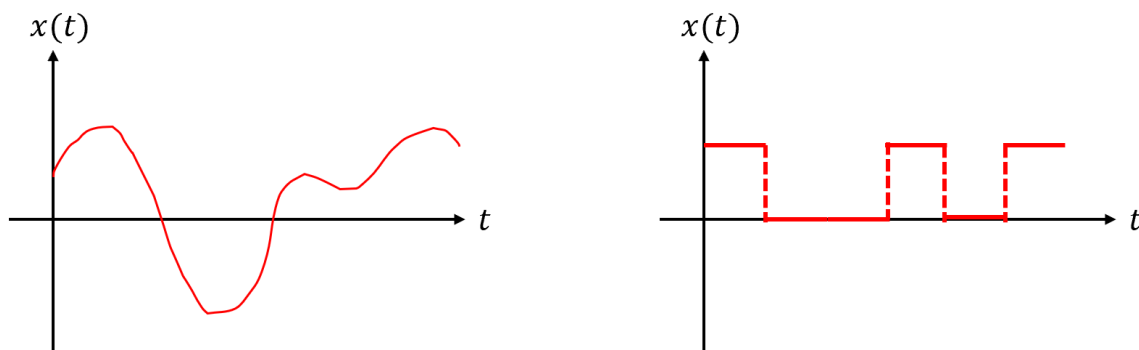
| 计数方式  | <b>attachInterrupt()</b><br>绑定中断处理函数 | 调用PCNT进行脉冲计数             |
|-------|--------------------------------------|--------------------------|
| 工作方式  | 每次边沿触发中断，中断处理函数<br>修改计数值             | PCNT硬件根据计数规则，自动对<br>边沿计数 |
| CPU压力 | 脉冲越多，中断越频繁，CPU越忙，<br>干扰其他程序执行        | CPU只需定时读取数据即可            |
| 抗毛刺   | 需要软件消抖                               | PCNT自带滤波器                |
| 适合场景  | 按键，旋转编码器，低频事件触发                      | 测速、测频率，高频事件触发            |

# 03 AD/DA转换

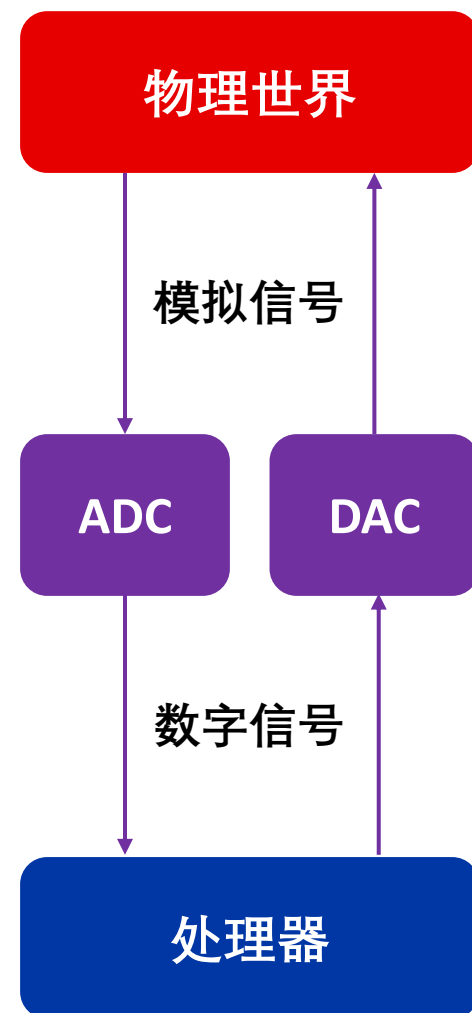


## 3.1 数字信号与模拟信号

在物理世界，我们面对的是各式各样的模拟信号，如温度、湿度、光强等等，它们在时间和幅度上都是连续的。而对于计算机而言，0和1的序列构成的离散的数字信号显然对存储、传输、运算、表示等各个环节都更加友好。



在嵌入式开发中，我们使用传感器获取物理世界中的模拟信号，并转换为数字信号，传递给处理器进行计算；完成计算后，再将数字信号转换为模拟信号，发送给执行器，从而对物理世界产生影响。完成这两次重要转换的，正是ADC和DAC。



## 3.2 数字模拟转换DAC

DAC的作用是将数字信号转换为模拟信号输出。目前常用的二进制电压型DAC有权电阻、权电容、R-2R等。

DAC的工作流程包括：

- **数字输入**：CPU送来二进制串行/并行数字量，如8bit 1100\_1001；
- **输入锁存**：防止输入信号抖动，保证DAC转换期间输入不变；

- **位权分配，求和运算**：数字量的每一位加权求和， $V_{out} = \frac{V_{ref}}{2^n} \sum_{i=0}^{n-1} D_i 2^i$

最低有效位  $V_{LSB}$   
= 参考电压 × 分辨率

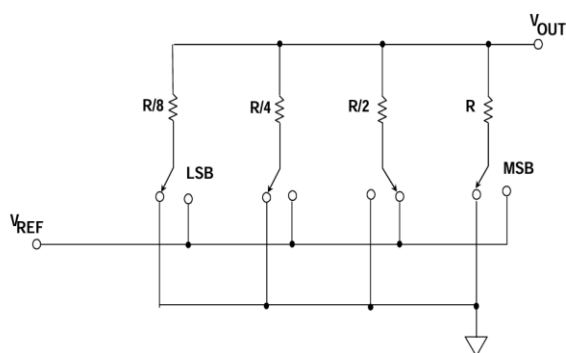
$$= \frac{V_{ref}}{256} (2^7 + 2^6 + 2^3 + 2^0) = \frac{201}{256} V_{ref}$$

- **输出整形与低通滤波**：滤去高频分量，使输出平滑。

## 3.2 数字模拟转换DAC

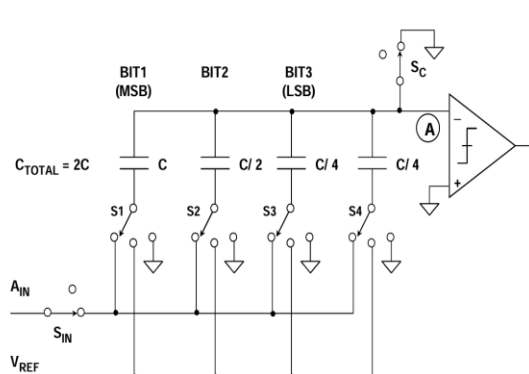
DAC的作用是将数字信号转换为模拟信号输出。目前常用的二进制电压型DAC有权电阻型、权电容型、R-2R网络型等。

- **位权分配，求和运算：** 数字量的每一位加权求和， $V_{out} = \frac{V_{ref}}{2^n} \sum_{i=0}^{n-1} D_i 2^i$



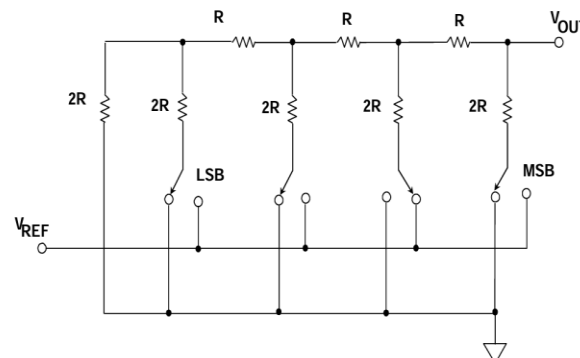
Adapted from: B. D. Smith, "Coding by Feedback Methods," Proceedings of the I. R. E., Vol. 41, August 1953, pp. 1053-1058

图1: 电压模式二进制加权电阻DAC



SWITCHES SHOWN IN TRACK (SAMPLE) MODE

图3: 逐次逼近型ADC中的电容二进制加权DAC



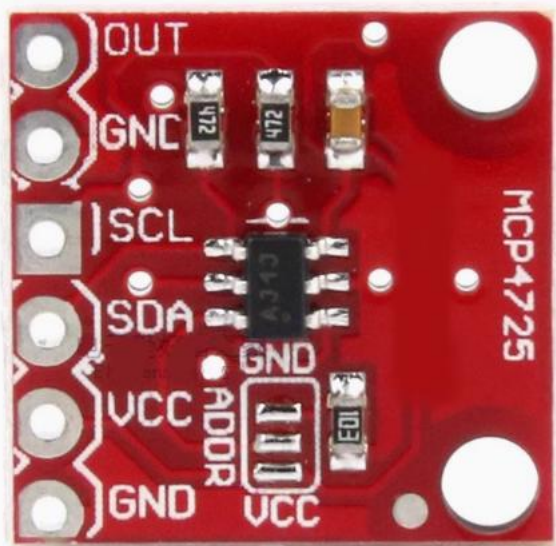
Adapted from: B. D. Smith, "Coding by Feedback Methods," Proceedings of the I. R. E., Vol. 41, August 1953, pp. 1053-1058

图5: 电压模式R-2R梯形电阻网络DAC

DAC输出的信号可能的取值只有  $2^n - 1$  个档位，依然是离散的。  
n越大，DAC的分辨率越高。

## 3.2 数字模拟转换DAC

ESP32S3没有可以输出模拟电压的自带DAC引脚，只能通过PWM波来进行模拟输出。如果需要输出模拟电压，可以借助其他元器件，如**MCP4725**模块。本实验了解即可，如有需要可自行学习。



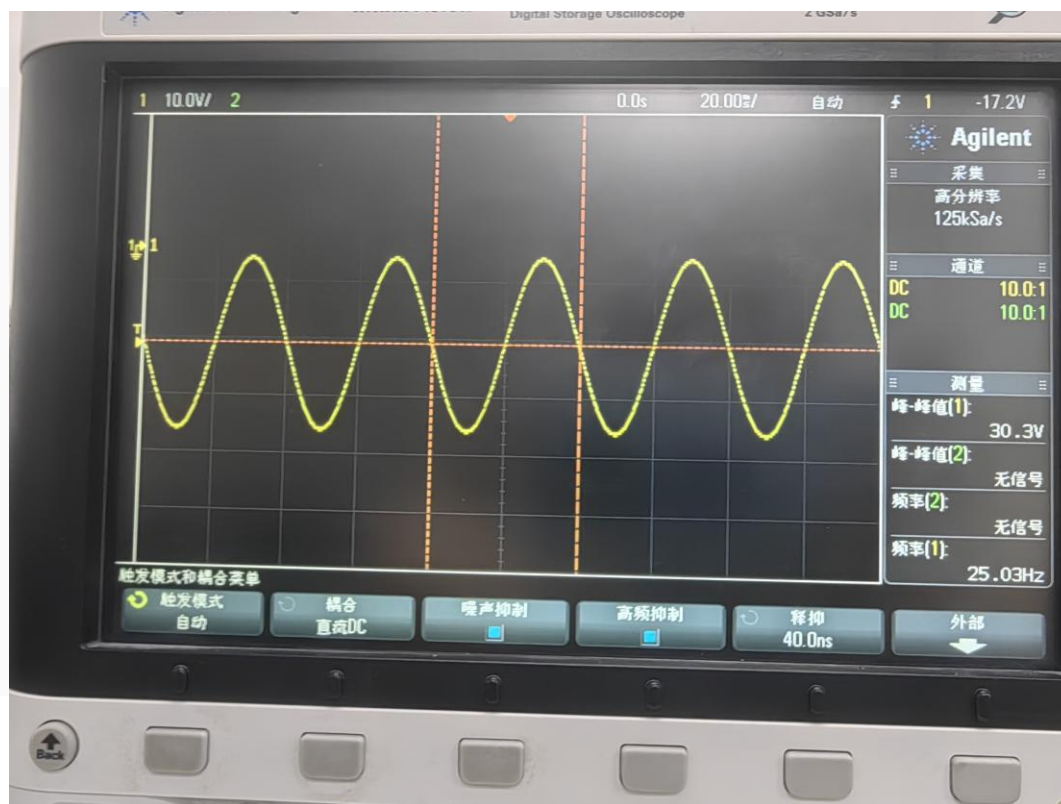
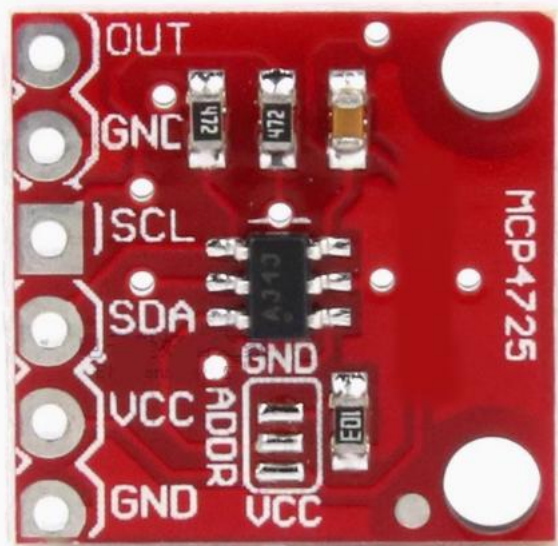
```
Adafruit_MCP4725 dac;
|
void setup() {
  Wire.begin(I2C_SDA, I2C_SCL);
  Wire.setClock(400000); // 400 kHz I2C

  float voltage = 2.25;
  uint16_t value = voltage * 4095 / DAC_VDD;
  dac.setVoltage(value, false);
}
```



## 3.2 数字模拟转换DAC

ESP32S3没有可以输出模拟电压的自带DAC引脚，只能通过PWM波来进行模拟输出。如果需要输出模拟电压，可以借助其他元器件，如**MCP4725**模块。本实验了解即可，如有需要可自行学习。



## 3.3 模拟数字转换ADC

ADC的作用是将连续的模拟输入信号转换为数字信号。

ADC的工作流程包括：

- **采样 (Sampling)**：按照固定周期，从输入信号中抽取瞬时电压值；
- **保持 (Hold)**：采样结束瞬间断开，储存读取结果，保证ADC转换期间输入不变；
- **量化 (Quantization)**：将模拟电压值映射到  $2^n - 1$  个档位，匹配最接近的一个；
- **编码 (Coding)**：将匹配到的离散电平转化为二进制码。

ESP32-S3的ADC资源：两个12位逐次逼近型ADC (**SAR ADC**)。

## 3.3 模拟数字转换ADC

ESP32-S3的ADC资源：两个12位逐次逼近型ADC (**SAR ADC**) 。

SAR ADC工作流程：

- S/H 采样保持；
- ADC内部的DAC，从最高位到最低位，输出试探电压；
- 比较器比较输入电压与试探电压，决定当前位置0或置1，n次比较完成编码；

举例： $n = 4$ ， $V_{ref} = 8$ ， $V_{in} = 5.6$

第3位，试探电压 $2^3 V_{LSB} = 4$ ，小于输入电压，置1；

第2位，试探电压 $(2^3 + 2^2) V_{LSB} = 6$ ，大于输入电压，置0；

第1位，试探电压 $(2^3 + 2^1) V_{LSB} = 5$ ，小于输入电压，置1；

第0位，试探电压 $(2^3 + 2^1 + 2^0) V_{LSB} = 5.5$ ，小于输入电压，置1；

最终输出编码 $D_{out} = 1011$ 。

## 3.3 模拟数字转换ADC

ESP32-S3的ADC资源：两个12位逐次逼近型ADC (**SAR ADC**)，可测量来自20个通道的模拟信号输入。

表 39.3-1. SAR ADC 的信号输入

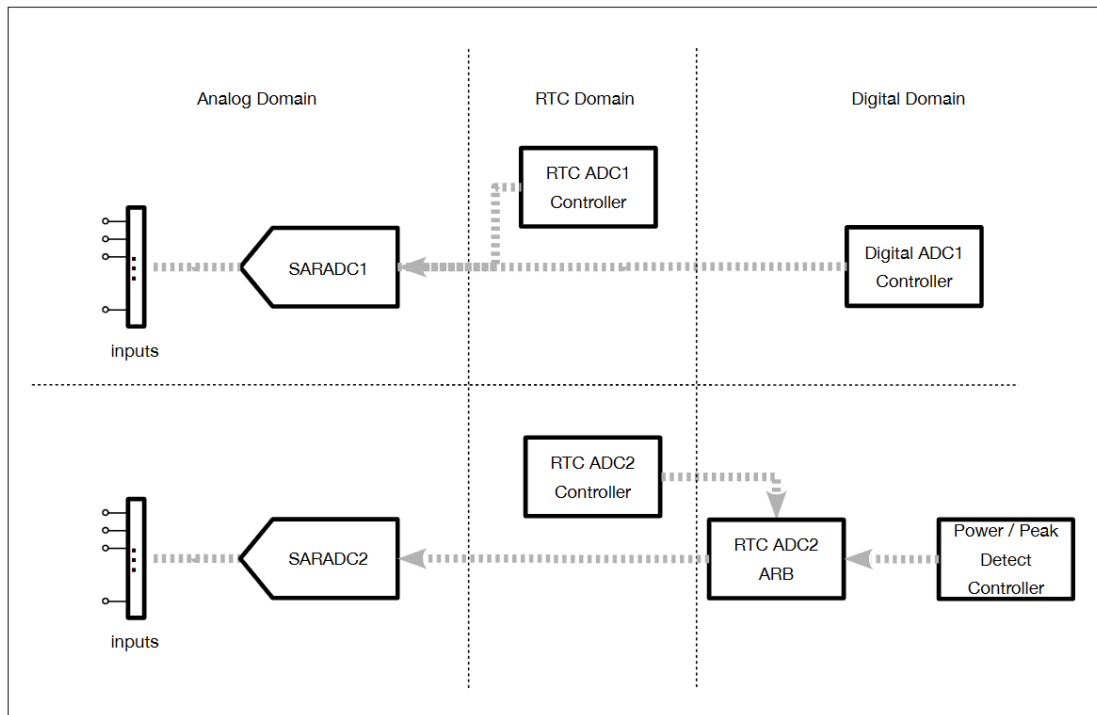


图 39.3-1. SAR ADC 概图

| 管脚/信号  | 通道 | ADC 选择   |
|--------|----|----------|
| GPIO1  | 0  | SAR ADC1 |
| GPIO2  | 1  |          |
| GPIO3  | 2  |          |
| GPIO4  | 3  |          |
| GPIO5  | 4  |          |
| GPIO6  | 5  |          |
| GPIO7  | 6  |          |
| GPIO8  | 7  |          |
| GPIO9  | 8  |          |
| GPIO10 | 9  |          |
| GPIO11 | 0  | SAR ADC2 |
| GPIO12 | 1  |          |
| GPIO13 | 2  |          |
| GPIO14 | 3  |          |
| GPIO15 | 4  |          |
| GPIO16 | 5  |          |
| GPIO17 | 6  |          |
| GPIO18 | 7  |          |
| GPIO19 | 8  |          |
| GPIO20 | 9  |          |

## 3.3 模拟数字转换ADC

ESP32-S3的ADC资源：两个12位逐次逼近型ADC (**SAR ADC**)，由四个专用控制器控制：

道的模拟信号输入

- 一个数字控制器DIG ADC1，支持高性能多通道扫描和DMA连续转换；

- 两个RTC控制器RTC ADC1/2，支持低功耗模式下工作和单次转换；

- 一个内部控制器PWDET，仅用于内部RF功率检测；

每个控制器配有单独的ADC Reader。

### • RTC ADC1/2 控制器：

- 支持单次转换
- 支持在低功耗模式下工作，如 Deep-sleep
- 可由 ULP 协处理器配置

### • DIG ADC1 控制器：

- 配有多通道扫描控制模块，支持多通道扫描模式
- 提供模式控制模块，支持单 SAR ADC 采样模式
- 在多通道扫描模式下，支持自定义扫描通道顺序
- 提供两个滤波器，滤波系数可配
- 支持阈值监控，采样值大于设置的高阈值或小于设置的低阈值将产生中断
- 支持 DMA

### • PWDET 控制器：用于 RF 功率检测（仅供内部使用）

| 引脚/信号 | 通道 | ADC 选择   |
|-------|----|----------|
|       | 0  | SAR ADC1 |
| GPIO2 | 1  |          |
| GPIO3 | 2  |          |
| GPIO4 | 3  |          |
| GPIO5 | 4  |          |
| GPIO6 | 5  |          |

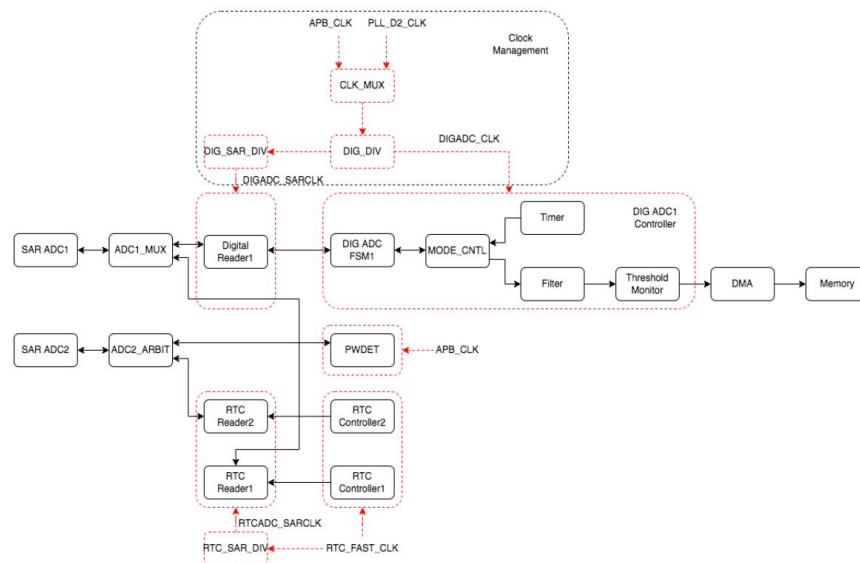
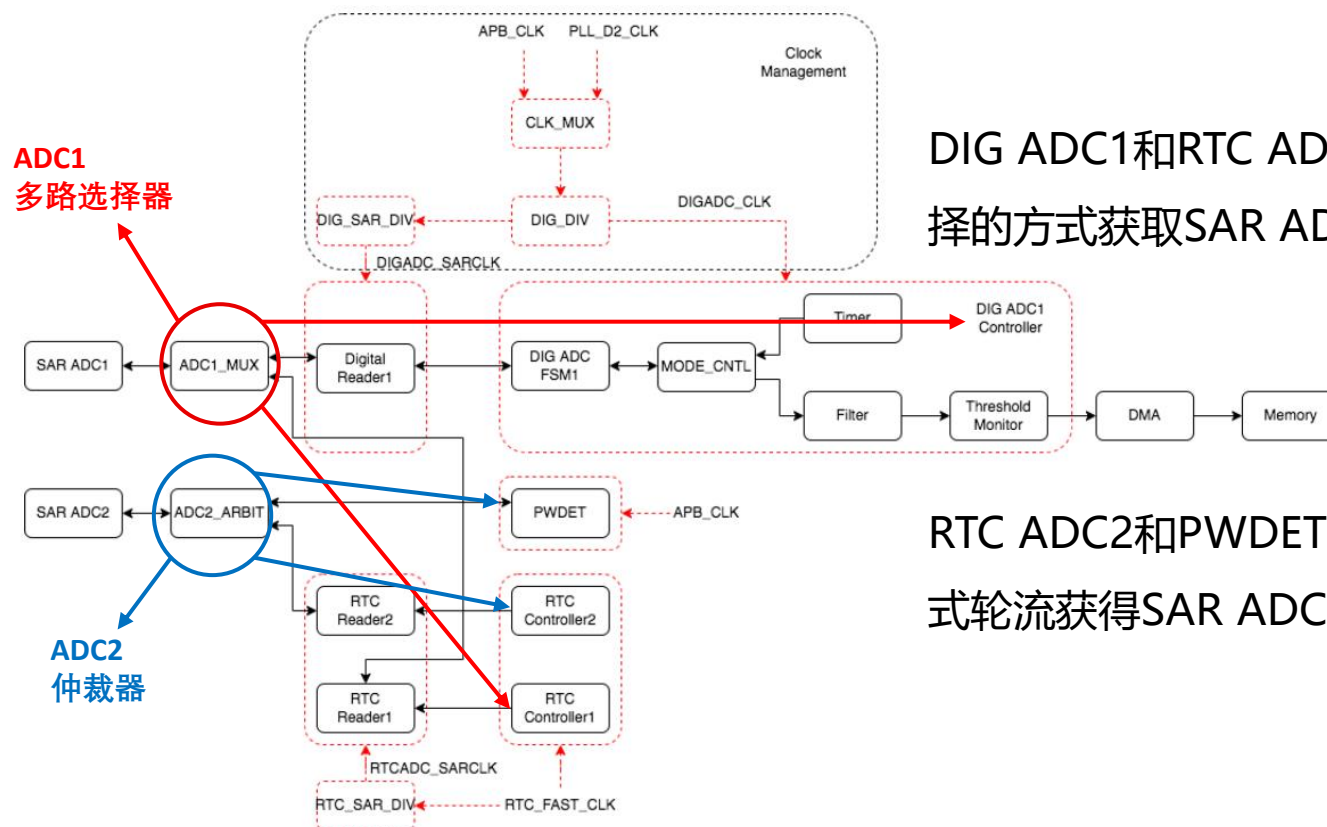


图 39.3-2. SAR ADC 的功能概况

## 3.3 模拟数字转换ADC

ESP32-S3的ADC资源：两个12位逐次逼近型ADC (**SAR ADC**)。



DIG ADC1和RTC ADC1通过软件选择的方式获取SAR ADC1的控制权；

RTC ADC2和PWDET通过仲裁的方式轮流获得SAR ADC2的控制权。

图 39.3-2. SAR ADC 的功能概况



## 3.3 模拟数字转换ADC

表 39.3-1. SAR ADC 的信号输入

| 管脚/信号  | 通道 | ADC 选择   |
|--------|----|----------|
| GPIO1  | 0  | SAR ADC1 |
| GPIO2  | 1  |          |
| GPIO3  | 2  |          |
| GPIO4  | 3  |          |
| GPIO5  | 4  |          |
| GPIO6  | 5  |          |
| GPIO7  | 6  |          |
| GPIO8  | 7  |          |
| GPIO9  | 8  |          |
| GPIO10 | 9  |          |
| GPIO11 | 10 | SAR ADC2 |
| GPIO12 | 11 |          |
| GPIO13 | 12 |          |
| GPIO14 | 13 |          |
| GPIO15 | 14 |          |
| GPIO16 | 15 |          |
| GPIO17 | 16 |          |
| GPIO18 | 17 |          |
| GPIO19 | 18 |          |
| GPIO20 | 19 |          |

在Arduino IDE中, 调用uint16\_t

**analogRead**(uint8\_t pin)函数, 可以读取目标引脚的模拟输入, 函数返回值是0~4095 ( $2^{12} - 1$ ) 之间的整数。

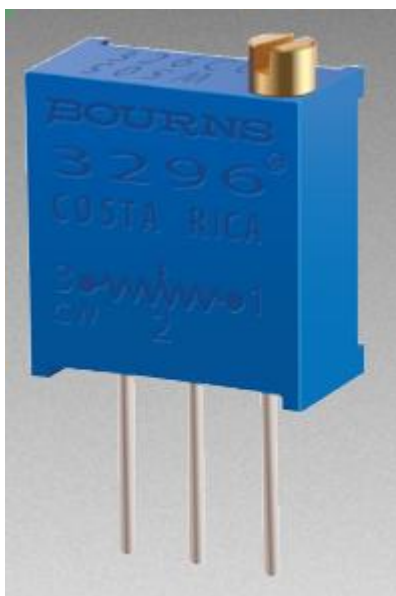
实际使用中, 如果需要同时使用多路模拟输入, 建议优先选择ADC1对应的GPIO1~10。

如果模拟输入的同时使用WiFi功能, 就不能使用ADC2对应的GPIO11~20。

```
function analogRead
→ uint16_t
Parameters:
• uint8_t pin
Get ADC value for pin
uint16_t analogRead(uint8_t pin)
analogRead(IN_PIN);
```

## 3.3 模拟数字转换ADC

实战练习：旋转电位器螺丝，读取中间引脚的模拟电压值。



理解为**滑动变阻器**，1、3是两个端点，2是中间的动点。

顺时针旋转螺丝，1、2之间阻值变大，2、3之间阻值变小。

## 3.3 模拟数字转换ADC

实战练习：旋转电位器螺丝，读取中间引脚的模拟电压值。

补充练习：对读取到的模拟电压值进行**滑动平均滤波**。

如何直观看数值变化？ **串口绘图仪**



## 3.3 模拟数字转换ADC

实战练习：使用ESP32S3片上的温度传感器，读取芯片的工作温度。  
该传感器将温度转换为模拟电压，经过SAR ADC转换为数字量。

其他适用于模拟输入的元器件：

- 带模拟口的光敏传感器
- 二维摇杆模块
- 模拟温度传感器
- 土壤湿度传感器
- .....

